

Diario de aula

None

Cristian Garcia

None

diario de aula

1. Presentación	6
2. 🚀 BIENVENIDA A AZAHARTECH	7
2.1 1.º DAM — Curso académico 2026/2027	7
2.2 🛠️ Sesión 1. El marco profesional y las reglas	7
2.3 ⚙️ Sesión 2. Pensamiento computacional y modelo IPO	8
3. 🧑‍🔧 Jornada de equipos y bolsa de proyectos	10
3.1 1.º DAM — Curso académico 2026/2027	10
4. Entornos de desarrollo (ED) — 50 minutos	10
4.1 🛠️ Sesión 1. El control de calidad y las herramientas	10
5. Proyecto intermodular (PI) — 50 minutos	11
5.1 🛠️ Sesión 2. La bolsa de proyectos	11
6. 🚀 Diario de clase: lunes, 14 de septiembre de 2026	14
6.1 1.º DAM — Curso académico 2026/2027	14
6.2 🛠️ Sesión 1. PR - Pensamiento computacional y variables numéricas primitivas	14
6.3 ⚙️ Sesión 2 (50 min). PR - Entrada de datos y aplicación al proyecto propio	15
6.4 🛠️ Sesión 3. ED - Sistema de información y ciclo de vida del software (SDLC)	16
6.5 🧑‍🔧 Sesión 4. Taller práctico de puesta a punto (OpenJDK 21 + IntelliJ IDEA)	16
6.6 📊 Estado al cierre de la jornada	17
7. 🚀 Diario de clase: martes, 15 de septiembre de 2026	18
7.1 1.º DAM — Curso académico 2026/2027	18
7.2 🛠️ Sesión 1. Caracteres (char), cadenas (String), booleanos y el buffer de Scanner	18
7.3 ⚙️ Sesión 2. Evolución acumulativa en el proyecto propio (v0.2)	20
7.4 📊 Estado al cierre de la jornada	20
8. 🚀 Diario de clase: lunes, 21 de septiembre de 2026	21
8.1 1.º DAM — Curso académico 2026/2027	21
8.2 🛠️ Sesión 1. PR - Asignación compuesta, incremento y mutabilidad de memoria	21
8.3 ⚙️ Sesión 2 (50 min). PR - Refactorización y aplicación al proyecto propio	21
8.4 🛠️ Sesión 3. ED - La mecánica interna de Git y la trazabilidad	22
8.5 🧑‍🔧 Sesión 4. Laboratorio práctico guiado: Flujo Git en IntelliJ IDEA	22
8.6 📊 Estado al cierre de la jornada	23
9. 🚀 Diario de clase: jueves, 17 de septiembre de 2026	24
9.1 1.º DAM — Curso académico 2026/2027	24
9.2 🛠️ Sesión 1. Estándares de calidad de código, autoformato e inspección en IntelliJ	24
9.3 ⚙️ Sesión 2. Consolidación y versionado visual del proyecto propio (v0.3)	27
9.4 📊 Estado al cierre de la jornada	28

10. 🚀	Diario de clase: viernes, 18 de septiembre de 2026	29
10.1	1.º DAM — Curso académico 2026/2027	29
10.2	🔗 Sesión 1. Metodología Scrum, estructura corporativa y primer repositorio en GitHub (ED)	29
10.3	⚙️ Sesión 2. De la idea al reto técnico: Análisis de necesidades, ODS y Sprint Backlog 1 (PI)	30
10.4	📊 Estado al cierre de la jornada	32
11. 🚀	Diario de clase: lunes, 21 de septiembre de 2026	33
11.1	1.º DAM — Curso académico 2026/2027	33
11.2	🔗 Sesión 1. PR - Asignación compuesta, incremento y mutabilidad de memoria	33
11.3	⚙️ Sesión 2 (50 min). PR - Refactorización y aplicación al proyecto propio	33
11.4	🔗 Sesión 3. ED - La mecánica interna de Git y la trazabilidad	34
11.5	🔧 Sesión 4. Laboratorio práctico guiado: Flujo Git en IntelliJ IDEA	34
11.6	📊 Estado al cierre de la jornada	34
12. 🚀	Diario de clase: martes, 22 de septiembre de 2026	36
12.1	1.º DAM — Curso académico 2026/2027	36
12.2	🔗 Sesión 1. PR - La división entera frente al operador residuo (%)	36
12.3	⚙️ Sesión 2 (50 min). PR - Aplicación al proyecto propio	36
12.4	📊 Estado al cierre de la jornada	37
13. 🚀	Diario de clase: miércoles, 23 de septiembre de 2026	38
13.1	1.º DAM — Curso académico 2026/2027	38
13.2	🔗 Sesión 1. PR - Precedencia de operadores y casting en Java	38
13.3	⚙️ Sesión 2 (50 min). PR - Aplicación al proyecto propio	39
13.4	📊 Estado al cierre de la jornada	39
14. 🚀	Diario de clase: jueves, 24 de septiembre de 2026	40
14.1	1.º DAM — Curso académico 2026/2027	40
14.2	🔗 Sesión 1. PR - Auditoría de calidad y consolidación v0.6	40
14.3	⚙️ Sesión 2 (50 min). PR - Cierre del ciclo y versionado	40
14.4	📊 Estado al cierre de la jornada	41
15. 🚀	Diario de clase: viernes, 25 de septiembre de 2026	42
15.1	1.º DAM — Curso académico 2026/2027	42
15.2	🔗 Sesión 1 (50 min). ED - El estándar Markdown en la ingeniería del software	42
15.3	⚙️ Sesión 2 (50 min). PI - Arquitectura de bloques y viabilidad	42
15.4	📊 Estado al cierre de la jornada	43
16. 🚀	Diario de clase: lunes, 28 de septiembre de 2026	44
16.1	1.º DAM — Curso académico 2026/2027	44
16.2	🔗 Sesión 1. PR - Constantes inmutables y literales tipados	44
16.3	⚙️ Sesión 2 (50 min). PR - Aplicación al proyecto propio	44
16.4	🔗 Sesión 3. ED - El cierre del Sprint y la calidad del repositorio	45
16.5	🔧 Sesión 4. Laboratorio práctico guiado. Auditoría de higiene y purga del repositorio	45

16.6	 Estado al cierre de la jornada	46
17.	 Diario de clase: martes, 29 de septiembre de 2026	47
17.1	1.º DAM — Curso académico 2026/2027	47
17.2	 Sesión 1. PR - Secuencias de escape en cadenas Java	47
17.3	 Sesión 2 (50 min). PR - Aplicación al proyecto propio	48
17.4	 Estado al cierre de la jornada	48
18.	 Diario de clase: miércoles, 30 de septiembre de 2026	49
18.1	1.º DAM — Curso académico 2026/2027	49
18.2	 Sesión 1. PR - Especificadores de formato en System.out.printf()	49
18.3	 Sesión 2 (50 min). PR - Aplicación al proyecto propio	50
18.4	 Estado al cierre de la jornada	50
19.	 Diario de clase: jueves, 1 de octubre de 2026	51
19.1	1.º DAM — Curso académico 2026/2027	51
19.2	 Sesión 1. PR - Documentación técnica y versión 1.0	51
19.3	 Sesión 2 (50 min). PR - Auditoría final del proyecto propio	51
19.4	 Estado al cierre de la jornada	52
20.	 Diario de clase: viernes, 2 de octubre de 2026	53
20.1	1.º DAM — Curso académico 2026/2027	53
20.2	 Sesión 1 (50 min). ED - Versionado semántico y etiquetas (TAGS)	53
20.3	 Sesión 2 (50 min). PI - Aspectos formales y técnica de comunicación oral	53
20.4	 Estado al cierre de la jornada y del Sprint 1	54
21.	 Diario de clase: lunes, 5 de octubre de 2026	55
21.1	1.º DAM — Curso académico 2026/2027	55
21.2	 Sesión 1. PR - Fundamentos de POO, instanciación y memoria (Stack vs. Heap)	55
21.3	 Sesión 2 (50 min). PR - Dojo de entrenamiento y katas de código	55
21.4	 Sesión 3. ED - La transformación del software y Bytecode universal	56
21.5	 Sesión 4. Laboratorio práctico: Compilación multi-entorno y katas	56
21.6	 Estado al cierre de la jornada	57
22.	 Diario de clase: martes, 6 de octubre de 2026	58
22.1	1.º DAM — Curso académico 2026/2027	58
22.2	 Sesión 1. PR - Variables primitivas frente a variables de referencia	58
22.3	 Sesión 2 (50 min). PR - Dojo de entrenamiento y katas de código	59
22.4	 Estado al cierre de la jornada	59
23.	 Diario de clase: miércoles, 7 de octubre de 2026	60
23.1	1.º DAM — Curso académico 2026/2027	60
23.2	 Sesión 1. PR - Constructores con parámetros y tipos de retorno	60
23.3	 Sesión 2 (50 min). PR - Dojo de entrenamiento y katas de código	61
23.4	 Estado al cierre de la jornada	61

24. 🚀	Diario de clase: jueves, 8 de octubre de 2026	62
24.1	1.º DAM — Curso académico 2026/2027	62
24.2	🕒 Sesión 1. PR - Métodos de transformación y extracción en la clase String	62
24.3	⚙️ Sesión 2 (50 min). PR - Dojo de entrenamiento y katas de código	63
24.4	📊 Estado al cierre de la jornada	63
25. 🚀	Diario de clase: viernes, 9 de octubre de 2026 (festivo, pero sirve para otros grupos)	64
25.1	1.º DAM — Curso académico 2026/2027	64
25.2	🕒 Sesión 1 (50 min). ED - Comparación de entornos de desarrollo	64
25.3	⚙️ Sesión 2 (50 min). PI - De la necesidad al catálogo de requisitos del sistema	64
25.4	📊 Estado al cierre de la jornada	65

1. Presentación

2. BIENVENIDA A AZAHARTECH

2.1 1.º DAM — Curso académico 2026/2027

Módulo: Programación (PR)

Fecha: Jueves, 10 de septiembre de 2026

Duración: 2 horas lectivas (100 min)

2.2 Sesión 1. El marco profesional y las reglas

2.2.1 1. Presentación de AzaharTech (10 min)

- Os incorporáis como **desarrolladores junior** en la consultora de software **AzaharTech** (Castellón de la Plana).
- Trabajamos de forma coordinada entre **Programación, Entornos de Desarrollo y Proyecto Intermodular**.

ECOSISTEMA		
PROGRAMACIÓN (PR) [8 h/semana] Construcción del código y lógica.	ENTORNOS DESARROLLO (ED) [3 h/semana] Git, compilación, IDE, testing y calidad.	PROYECTO INTERMODULAR [1 h/semana] Gestión del reto, IA, viabilidad y defensas.

2.2.2 2. El caso guía: Sistema de control de asistencia con QR para el IES El Caminàs (5 min)

- **Proyecto.** Sistema de control de asistencia mediante códigos QR dinámicos mostrados en la pantalla del vestíbulo.
- **Propósito.** Es el proyecto de referencia que utilizará el docente para modelar los contenidos en la pizarra y en pantalla.
- **Importante.** No es el proyecto que evaluará al alumno. Cada grupo elegirá mañana su propio reto de la **Bolsa de Proyectos Oficial**.

 **Nota:** La app del instituto es el espejo donde mirar cómo se programa, pero cada estudiante desarrollará su propia aplicación a partir del lunes.

2.2.3 3. Las dos fases del curso y la estructura en Git (10 min)

Para garantizar un aprendizaje real y sin brechas:

Periodo	Fase	Aula	Dinámica
1.º Trimestre (Sep - Dic)	Fase I «Desarrollador junior»	Trabajo individual	• Código 100 % propio. • Exámenes y entregas individuales. • Base sólida para todos.
2.º Trimestre (Ene - May)	Fase II «Equipo de consultoría»	Trabajo en equipo	• Metodología Scrum real. • Reparto de tareas y backlog. • Repositorio colaborativo.

¿Por qué existe la carpeta de equipo desde hoy?

```

azahartech/
├── equipo-01/
│   ├── apellidos-nombre/
│   │   └── ed/
└── <-- Identificador asignado desde el primer día
    <-- Tu espacio personal e intransferible (T1)
    <-- Entornos de Desarrollo
  
```

```
pr/      <-- Programación
pi/      <-- Proyecto Intermodular
```

* En enero tu equipo añadirá una carpeta común `proyecto-conjunto/` sin tener que renombrar carpetas ni romper rutas en IntelliJ y Git.

2.2.4 4. Actividad 1. Cuestionario de diagnóstico (10 min)

📱 **Acceso al cuestionario:** Entra en Aules y rellena el cuestionario

2.2.5 5. Dudas y debate abierto (15 min)

- Turno de preguntas sobre la evaluación, el material necesario y el ritmo de trabajo del curso.

2.3 ⚙️ Sesión 2. Pensamiento computacional y modelo IPO

2.3.1 1. Actividad 2. «El robot ciego» (20 min)

Dinámica por parejas para experimentar qué es un algoritmo y por qué ocurren los errores de programación.



Reglas de la dinámica:

1. Por parejas, un estudiante asume el rol de **programador** y el otro de **robot**.
2. El programador escribe en un papel una secuencia de **exactamente 5 órdenes** para lograr que el robot coja un bolígrafo de la mesa y lo deposite dentro de una mochila sin tirarlo.
3. El robot ejecuta las órdenes de forma ciega y literal (si la orden es "*mueve el brazo hacia adelante*" y no indica cuántos centímetros, el robot se mueve hasta chocar).
4. **Puesta en común (5 min):** ¿Quién ha conseguido el objetivo al primer intento? ¿Por qué la máquina ha fallado cuando la instrucción parecía obvia?

💡 **Conclusión.** Un ordenador no interpreta intenciones; solo ejecuta instrucciones exactas sobre datos. Un algoritmo debe ser **preciso, ordenado y no ambiguo**.

Exact Instructions Challenge - video de Josh Darnit <https://www.youtube.com/watch?v=FN2RM-CHkuI>

2.3.2 2. El modelo universal de la informática: IPO (10 min)

Todo sistema informático responde a este esquema de tres fases consecutivas:



- | | | |
|--------------|---------------------|-----------------|
| • Sensores | • Lógica de control | • Archivos / BD |
| • Escáner QR | • Transformaciones | • Actuadores |

• **Ejemplo del IES El Caminàs:**

- *Entrada:* DNI del alumno + hora de lectura del QR.
- *Procesamiento:* comprobar si la hora es posterior a las 08:05 h y calcular los minutos de retraso.
- *Salida:* mensaje en pantalla («Puntual» en verde o «Retraso» en naranja) + registro en la base de datos.

2.3.3 3. Actividad 3. Descomposición en papel (15 min)

Dinámica individual con puesta en común (think-pair-share).

1. **Piensa en solitario (5 min).** Elige un sistema que utilices a diario (cajero automático, máquina de café, barrera de parking, inicio de sesión en una app) y dibuja en tu libreta las 3 cajas:
 - ¿Qué datos introduce el usuario?
 - ¿Qué cálculos o verificaciones realiza la máquina internamente?
 - ¿Qué resultado devuelve?
2. **Compara con el compañero (5 min).** Intercambia la libreta con el compañero de al lado y comprueba su diagrama:
 - «¿Qué ocurre si el usuario teclea mal el PIN?»
 - «¿Has indicado si el dato es un número o un texto?»
3. **Puesta en común en la pizarra (5 min).** Dos alumnos voluntarios dibujan su esquema y el resto del aula propone casos límite.

2.3.4 4. Cierre del día 10 (5 min)

ESTADO AL CIERRE DE HOY
[] Presentado el funcionamiento y simulación de AzaharTech.
[] Aclarada la fase individual del Trimestre 1 en el aula.
[] Cuestionario de diagnóstico completado por todo el grupo.
[] Asimilado el concepto de algoritmo y el modelo IPO.

||

3. 🛠️ Jornada de equipos y bolsa de proyectos

3.1 1.º DAM — Curso académico 2026/2027

Módulos: Entornos de desarrollo (ED) + Proyecto intermodular (PI)

Fecha: Viernes, 11 de septiembre de 2026

Duración: 2 sesiones lectivas (100 min)

4. Entornos de desarrollo (ED) — 50 minutos

4.0.1 La puesta a punto del taller digital de AzaharTech

4.1 🕒 Sesión 1. El control de calidad y las herramientas

4.1.1 1. El papel de entornos de desarrollo en AzaharTech (5 min)

- En programación construimos la lógica del software.
- En **entornos de desarrollo** aseguramos la **calidad de ingeniería**:
- Manejo del compilador y del IDE profesional (**IntelliJ IDEA**).
- Uso de sistemas de control de versiones y trazabilidad con **Git** y **GitHub**.
- Higiene del repositorio y entrega formal con etiquetas de versión (**Git tags**).

4.1.2 2. Publicación oficial de los equipos de AzaharTech (10 min)

Equipos equilibrados y heterogéneos confeccionados a partir del cuestionario de diagnóstico de ayer.

Equipo de trabajo	Integrantes asignados	Identificador Git oficial
Equipo 01	[Estudiante 1] · [Estudiante 2] · [Estudiante 3] · [Estudiante 4]	equipo-01
Equipo 02	[Estudiante 5] · [Estudiante 6] · [Estudiante 7] · [Estudiante 8]	equipo-02
Equipo 03	[Estudiante 9] · [Estudiante 10] · [Estudiante 11] · [Estudiante 12]	equipo-03
Equipo 04	[Estudiante 13] · [Estudiante 14] · [Estudiante 15] · [Estudiante 16]	equipo-04

📢 **Nota.** Al proyectar la lista de equipos, resolveremos incidencias puntuales y confirmaremos que todo el grupo conoce su identificador.

4.1.3 3. Actividad 1. Comprobación del taller digital (20 min)

Comprobación práctica en los puestos de trabajo para no perder tiempo el lunes.

CHECKLIST TÉCNICO EN EL PUESTO
1. Abrir la terminal (bash) en Linux Mint y ejecutar: java -version (comprobar si el equipo del aula tiene Java instalado)
2. Acceder a GitHub estudiante*:

Crear cuenta personal de estudiante o verificar credenciales.

3. Descargar instaladores:
- IntelliJ IDEA Community Edition
 - PseInt

*<https://docs.github.com/es/education/about-github-education/github-education-for-students/apply-to-github-education-as-a-student>

 **Nota.** El profesor irá por el aula resolviendo dudas de cuentas de GitHub, comprobando versiones de Java y ayudando en las descargas.

4.1.4 4. Simulación de la jerarquía de carpetas oficial (10 min)

Apunta en tu libreta la ruta exacta que crearás el lunes en la primera sesión práctica de Git:

```
azahartech/
├── equipo-01/
│   └── apellidos-nombre/
│       ├── ed/
│       ├── pr/
│       └── pi/
└── <-- El identificador que te ha tocado hoy
    <-- Tus apellidos y nombre (sin espacios)
    <-- Entornos de desarrollo
    <-- Programación
    <-- Proyecto intermodular
```

- **Regla de estilo.** Todo en minúsculas, sin espacios, sin acentos y sin caracteres especiales.

4.1.5 5. Dudas técnicas y cierre del bloque de ED (5 min)

- Comprobación de que todo el alumnado tiene cuenta de GitHub operativa y conoce su identificador de equipo.

5. Proyecto intermodular (PI) — 50 minutos

5.0.1 Elección del proyecto

5.1 Sesión 2. La bolsa de proyectos

5.1.1 1. ¿Qué es el proyecto intermodular? (10 min)

- Es el módulo que conecta la técnica con el mundo laboral real.
- **El nexa entre los módulos:**
- En **PR** programáis los algoritmos y la lógica.
- En **ED** gestionáis el repositorio, el versionado y la calidad.
- En **PI** analizáis al cliente, gestionáis el backlog y defendéis el software ante el público.
- **Las sesiones de los viernes:** Espacio reservado para tutorización, uso ético de la IA (*prompt log*), arquitectura y preparación de la *sprint review*.

5.1.2 2. Catálogo de proyectos (15 min)

Área 1: Ciberseguridad

- **P1. Simulador de phishing:** Juego interactivo de detección de mensajes fraudulentos; clasificación por tipología (email, SMS, red social) con pistas, usuarios, estadísticas e historial.
- **P2. Bóveda de contraseñas:** Gestor de credenciales con evaluación de fortaleza mediante políticas de seguridad (básica, estricta, corporativa), caducidad y almacén seguro.

Área 2: Python

- **P3. Calculadora paso a paso:** Evaluador de expresiones matemáticas con desglose paso a paso, tokenización, precedencia, visor del proceso e histórico.
- **P4. Generador de datos de prueba:** Herramienta de síntesis de datasets sintéticos a partir de plantillas con campos polimórficos y exportación a ficheros CSV.

Área 3: Videojuegos y realidad virtual

- **P5. Aventura conversacional:** Motor de narrativa interactiva ramificada con toma de decisiones, escenas, inventario, personajes (aliado, enemigo, neutral) y guardado.
- **P6. Simulador de físicas 2D:** Entorno de simulación cinemática con detección de colisiones, múltiples bolas con comportamientos físicos y telemetría.

Área 4: Productos software en contenedores

- **P7. Reparto de contenedores en nodos:** Planificador de asignación de carga en clúster según capacidad de CPU y memoria, nodos especializados e interfaz con barras de ocupación.
- **P8. Catálogo de servicios + generador de config:** Asistente paso a paso (*wizard*) para parametrizar servicios (web, BD, worker), catálogo e historial de despliegues.

Área 5: Inteligencia artificial y big data

- **P9. Motor de recomendación:** Sistema basado en afinidad entre gustos y catálogos de ítems categorizados (película, música, libro) con cálculo de coincidencia por etiquetas.
- **P10. Analizador de reseñas:** Analizador de sentimiento léxico sobre comentarios con diccionarios de palabras clave y estadísticas por producto.

Área 6: Recursos y servicios en la nube

- **P11. Cotizador cloud:** Presupuestador interactivo de infraestructura cloud con cálculo de costes según recurso (cómputo, almacenamiento, red) y comparación de ofertas.
- **P12. Dashboard de monitorización:** Panel de control visual para métricas y telemetría de sistemas, lectura de sensores (CPU, memoria, red) y umbrales de alerta.

5.1.3 3. Actividad 2. Elección y consenso en el equipo (15 min)

FICHA DE ELECCIÓN DE RETO (1 POR EQUIPO)	
Equipo:	_____
Integrantes:	_____
1. ^a Opción prioritaria:	_____
2. ^a Opción de reserva:	_____
3. ^a Opción de reserva:	_____

- Los 4 integrantes de cada equipo se reúnen en el aula para debatir vocaciones e intereses.
- Seleccionan sus 3 opciones y entregan la ficha física al profesor.

 **Nota.** El profesor os explicará las dudas técnicas de los proyectos y recogerá las fichas con vuestras preferencias.

5.1.4 4. Adjudicación en pizarra (5 min)

- El profesor adjudica los proyectos en directo garantizando variedad temática en el aula.

- Se registra la asignación oficial en el acta docente (ej. *Equipo 01 → P11. Cotizador cloud; Equipo 02 → P1. Simulador de phishing...*).
-

5.1.5 5. Cierre del día 11 (5 min)

ESTADO AL CIERRE DE HOY
☐ Identificador de equipo asignado (por ejemplo: equipo-02).
☐ Proyecto adjudicado de la bolsa de proyectos.
☐ Comprobación técnica de puesto completada (terminal y GitHub).
☐ Ruta de carpetas anotada para el inicio de las clases.

6. 🚀 Diario de clase: lunes, 14 de septiembre de 2026

6.1 1.º DAM — Curso académico 2026/2027

Módulos: Programación (PR) + Entornos de desarrollo (ED)

Fecha: Lunes, 14 de septiembre de 2026

Duración: 4 horas lectivas (200 min)

6.2 🧭 Sesión 1. PR - Pensamiento computacional y variables numéricas primitivas

Balance de tiempo: **Docente: 15 min | Estudiante: 35 min**

6.2.1 1. Contexto profesional en AzaharTech (5 min)

- Arranque del caso guía: sistema de control de acceso por QR para el **IES El Caminàs**.
- Laia Claramunt presenta el reto del día: el terminal debe capturar y registrar en la memoria RAM su identificador y la lectura térmica del vestíbulo.

6.2.2 2. Micro-exposición docente (10 min)

- Qué es un algoritmo: secuencia ordenada Entrada $\rightarrow$ Proceso $\rightarrow$ Salida (modelo IPO).
- Anatomía básica de un archivo Java:

```
public class ControlAccesoQR {  
    public static void main(String[] args) {  
        // Instrucciones secuenciales  
    }  
}
```

- Variables en memoria RAM: identificador en *camelCase* y tipo de dato.
- Tipos numéricos primitivos: `int` (enteros de 32 bits) y `double` (reales con decimales de 64 bits).

6.2.3 3. Andamiaje guiado: PSeInt → Java (25 min)

Los estudiantes diseñan en PSeInt y traducen inmediatamente a Java en IntelliJ:

• **PSeInt (pr/pseudocodigo/ControlAccesoQR.psc - v0.1):**

```
Algoritmo ControlAccesoQR
    Definir terminalId Como Entero
    Definir tempVestibulo Como Real

    Escribir "ID del terminal:"
    Leer terminalId
    Escribir "Temperatura del sensor (°C):"
    Leer tempVestibulo

    Escribir "Terminal configurado: #", terminalId
    Escribir "Lectura térmica: ", tempVestibulo, " °C"
FinAlgoritmo
```

• **Java (pr/src/ControlAccesoQR.java - v0.1):**

```
import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;

        System.out.print("ID del terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura del sensor (°C): ");
        tempVestibulo = teclado.nextDouble();

        System.out.println("Terminal configurado: #" + terminalId);
        System.out.println("Lectura térmica: " + tempVestibulo + " °C");

        teclado.close();
    }
}
```

6.2.4 4. Puesta en común (10 min)

- Resolución de dudas en pantalla: la importancia del punto y coma (;), coincidencia exacta del nombre de archivo y clase, y el cierre de llaves { }.

6.3 Sesión 2 (50 min). PR - Entrada de datos y aplicación al proyecto propio

Balance de tiempo: **Docente: 10 min | Estudiante: 40 min**

6.3.1 1. Pautas técnicas para el proyecto propio (10 min)

- El docente explica cómo extrapolar las variables numéricas a los cuatro proyectos del aula:
- En Aventura conversacional: puntos de vida (int puntosVida) y multiplicador de daño (double factorDano).
- En Motor de recomendación: identificador de ítem (int idItem) y valoración media (double valoracionMedia).
- En Simulador de físicas 2D: número de colisiones (int totalColisiones) y masa del cuerpo (double masaKg).
- En Bóveda de contraseñas: días para la caducidad (int diasCaducidad) y porcentaje de fortaleza (double nivelSeguridad).

6.3.2 2. Trabajo del estudiante en el proyecto propio (35 min)

- Cada estudiante crea en su espacio personal (azahartech/nombre-equipo/apellidos-nombre/pr/):

- a. pr/pseudocodigo/MiProyecto.psc (versión v0.1).
- b. pr/src/MiProyecto.java (versión v0.1).

- Declara al menos una variable `int` y una variable `double` representativas de su proyecto.
- Captura datos mediante `Scanner` y muestra la confirmación por consola.
- El docente circula por el aula resolviendo incidencias individuales de tipado y lectura.

6.3.3 3. Cierre de la sesión de programación (5 min)

- Comprobación en consola de que el código compila y ejecuta sin errores (`Ctrl + Shift + F10`).

6.4 Sesión 3. ED - Sistema de información y ciclo de vida del software (SDLC)

Balance de tiempo: **Docente: 15 min | Estudiante: 35 min**

6.4.1 1. Micro-exposición docente (15 min)

- **Programa vs. Sistema de información:** Un programa es solo una pieza; un Sistema de Información (SI) articula 5 componentes: Hardware, Software, Datos, Personas y Procesos.
- **Ciclo de vida del software (SDLC):** Análisis $\rightarrow$ Diseño $\rightarrow$ Codificación $\rightarrow$ Pruebas $\rightarrow$ Despliegue $\rightarrow$ Mantenimiento.
- **La regla del coste de corrección:** Un error no detectado en el análisis cuesta hasta 100 veces más corregirlo en producción.

LOS 5 COMPONENTES DE UN SISTEMA DE INFORMACIÓN	
1. Hardware	Dispositivos físicos (pantallas, terminales, CPU).
2. Software	Programas y backend en Java que desarrollamos.
3. Datos	Materia prima (censos, registros, marcas de tiempo).
4. Personas	Usuarios finales, administradores y operadores.
5. Procesos	Reglas operativas y normativas de la organización.

6.4.2 2. Actividad. Identificación de componentes (20 min)

- Cada estudiante redacta en su libreta o en un archivo borrador el análisis de los 5 componentes aplicados al proyecto que eligió de la bolsa de proyectos.
- Debe identificar:
 - Al menos 2 dispositivos de hardware requeridos.
 - Qué datos en bruto manejará su aplicación.
 - Dos perfiles de personas (usuarios) distintos.
 - Un proceso de negocio claro que el software automatizará.

6.4.3 3. Puesta en común (15 min)

- Debate guiado: tres alumnos exponen sus componentes y el grupo analiza si se ha confundido algún dato con un proceso o un componente software con hardware.

6.5 Sesión 4. Taller práctico de puesta a punto (OpenJDK 21 + IntelliJ IDEA)

Balance de tiempo: **Docente: 15 min | Estudiante: 35 min**

6.5.1 1. Fundamento técnico del entorno (15 min)

- Cómo funciona Java: Código fuente (`.java`) $\rightarrow$ Compilador `javac` $\rightarrow$ Bytecode (`.class`) $\rightarrow$ Máquina Virtual de Java (JVM).

- Principio WORA (*Write Once, Run Anywhere*).
- Anatomía de un IDE profesional: editor inteligente, gestor de compilación, depurador y control de versiones.

```
[ Código fuente (.java) ] —(javac)→ [ Bytecode (.class) ] —(JVM)→ [ Procesador / SO ]
```

6.5.2 2. Taller guiado (30 min)

- Cada estudiante realiza en su puesto de trabajo los siguientes pasos técnicos:

a. Abre la terminal del sistema y verifica la instalación del compilador:

```
java --version
javac --version
```

b. Abre **IntelliJ IDEA Community Edition**:

- Configura la codificación global del proyecto en `UTF-8` (*Settings -> File Encodings*).
- Asocia el SDK oficial a **OpenJDK 21**.

c. Abre el archivo de tu proyecto y verifica que compila y ejecuta en la consola integrada.

d. Realiza una captura completa de pantalla mostrando IntelliJ, el código y la terminal con `java --version`.

e. Guarda la imagen con el nombre exacto `entorno.png` en su carpeta temporal (se moverá a su ruta definitiva en la sesión de Git).

6.5.3 3. Cierre y verificación de puestos (5 min)

- El docente comprueba visualmente que todos los puestos tienen OpenJDK 21 verificado y la captura `entorno.png` lista.

6.6 Estado al cierre de la jornada

ESTADO AL CIERRE DEL DÍA	
[]	Ecosistema de desarrollo verificado con OpenJDK 21 e IntelliJ.
[]	Evidencia 1 de Entornos de desarrollo generada (entorno.png).
[]	Comprendida la estructura de un programa Java y el modelo IPO.
[]	Asimilados los tipos primitivos int y double en memoria RAM.
[]	Primera versión (v0.1) del proyecto propio diseñada y compilada.

7. 🚀 Diario de clase: martes, 15 de septiembre de 2026

7.1 1.º DAM — Curso académico 2026/2027

Módulo del día: Programación (PR) **Fecha:** Martes, 15 de septiembre de 2026

Duración: 2 horas lectivas (100 min)

7.2 🕒 Sesión 1. Caracteres (`char`), cadenas (`String`), booleanos y el buffer de Scanner

Balance de tiempo: **Docente: 15 min | Estudiante: 35 min**

7.2.1 1. Contexto profesional en AzaharTech (5 min)

- Pau Ferrer ejecuta la versión v0.1 de ayer: el sistema registra el terminal `101` y la temperatura `21.5`, pero no tiene forma de identificar al estudiante que escanea.
- Laia Claramunt plantea el reto: necesitamos capturar el nombre de la persona, su DNI, el perfil de usuario y saber si está entrando o saliendo.

7.2.2 2. Micro-exposición docente: Memoria alfanumérica y lógica (10 min)

- **Carácter individual (`char`):** Ocupa 16 bits (Unicode) y se delimita siempre con comillas simples (`'A'`).
- **Cadena de texto (`String`):** Objeto que almacena secuencias de texto de cualquier longitud, delimitado con comillas dobles (`"Laura Vidal"`).
- **Valor lógico (`boolean`):** Solo admite dos literales reservados: `true` o `false`.
- **La trampa del buffer de teclado en `Scanner`:**
- Al teclear un número y pulsar *Enter*, el método `nextInt()` lee el número pero deja el salto de línea (`\n`) flotando en el buffer.
- Si a continuación se invoca `nextLine()`, este lee el `\n` residual y cree que el usuario introdujo una línea vacía.
- **Solución:** Intercalar una llamada de limpieza previa con `teclado.nextLine();`.

GESTIÓN DEL BUFFER EN SCANNER

1. <code>teclado.nextInt();</code>	→ Lee el número, deja el <code>'\n'</code> en buffer.
2. <code>teclado.nextLine();</code>	→ Limpieza obligatoria: consume el <code>'\n'</code> .
3. <code>teclado.nextLine();</code>	→ Lee ahora sí el texto real del usuario.

7.2.3 3. Andamiaje guiado: Evolución a ControlAccesoQR v0.2 (25 min)

Los estudiantes abren el archivo de ayer y lo amplían directamente en PSeInt y en IntelliJ:

- **PSeInt (pr/pseudocodigo/ControlAccesoQR.psc - v0.2):**

```

Algoritmo ControlAccesoQR
    Definir terminalId Como Entero
    Definir tempVestibulo Como Real

    Definir nombrePersona, dniPersona Como Cadena
    Definir perfilPersona Como Caracter
    Definir esEntrada Como Logico

    Escribir "ID del terminal:"
    Leer terminalId
    Escribir "Temperatura del sensor (°C):"
    Leer tempVestibulo

    Escribir "DNI de la persona:"
    Leer dniPersona
    Escribir "Nombre completo:"
    Leer nombrePersona
    Escribir "Perfil de acceso (E = Estudiante, D = Docente, V = Visita):"
    Leer perfilPersona

    esEntrada <- Verdadero

    Escribir "Terminal configurado: #", terminalId
    Escribir "Sensor termico: ", tempVestibulo, " °C"
    Escribir "Persona: ", nombrePersona, " (DNI: ", dniPersona, ")"
    Escribir "Perfil: ", perfilPersona
    Escribir "Sentido del paso: Entrada (", esEntrada, ")"
FinAlgoritmo
  
```

- **Java (pr/src/ControlAccesoQR.java - v0.2):**

```

import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;

        String dniPersona;
        String nombrePersona;
        char perfilPersona;
        boolean esEntrada;

        System.out.print("ID del terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura del sensor (°C): ");
        tempVestibulo = teclado.nextDouble();
        teclado.nextLine();

        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();

        System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita):");
        perfilPersona = teclado.next().charAt(0);

        esEntrada = true;

        System.out.println("Terminal configurado: #" + terminalId);
        System.out.println("Sensor termico: " + tempVestibulo + " °C");
        System.out.println("Persona: " + nombrePersona + " (DNI: " + dniPersona + ")");
        System.out.println("Perfil: " + perfilPersona);
        System.out.println("Sentido del paso: Entrada (" + esEntrada + ")");

        teclado.close();
    }
}
  
```

7.2.4 4. Provocando el error del buffer (10 min)

- Cada estudiante comenta voluntariamente la línea de limpieza `// teclado.nextLine();` en su código y ejecuta el programa.

- Comprueban en pantalla cómo el programa se salta la lectura del nombre. Vuelven a descomentar la línea para interiorizar la solución.

7.3 Sesión 2. Evolución acumulativa en el proyecto propio (v0.2)

Balance de tiempo: **Docente: 10 min | Estudiante: 40 min**

7.3.1 1. Pautas técnicas de adaptación a los proyectos de la bolsa (10 min)

- El docente explica cómo trasladar los tipos alfanuméricos (String, char) y booleanos (boolean) a las cuatro temáticas:
- En Aventura conversacional: nombre del héroe (String), clase de personaje 'G', 'M', 'P' (char) y si la partida está activa (boolean estaVivo).
- En Motor de recomendación: título de la película/libro (String), tipo de contenido 'P', 'M', 'L' (char) y si está marcado como favorito (boolean esFavorito).
- En Simulador de físicas 2D: etiqueta del cuerpo (String), tipo de partícula 'N', 'P', 'R' (char) y si tiene la gravedad activada (boolean gravedadActiva).
- En Bóveda de contraseñas: nombre del servicio o web (String), nivel de política 'B', 'E', 'C' (char) y si requiere doble factor (boolean requiere2FA).

7.3.2 2. Trabajo autónomo en puesto individual (35 min)

- Cada estudiante abre su archivo único en IntelliJ (`pr/src/MiProyecto.java`) y en PSeInt (`pr/pseudocodigo/MiProyecto.psc`).
- Evoluciona el código de la v0.1 a la **v0.2**:
 - a. Mantiene las variables numéricas creadas ayer.
 - b. Declara al menos una variable `String`, una `char` y una `boolean`.
 - c. Aplica la captura con `Scanner` asegurando la limpieza del buffer.
 - d. Muestra un resumen claro en consola.
- El docente circula resolviendo dudas de lectura de caracteres individuales y variables lógicas.

7.3.3 3. Cierre y verificación técnica (5 min)

- Comprobación en consola de que el programa permite teclear nombres con espacios sin fallos de compilación.

7.4 Estado al cierre de la jornada

```

ESTADO AL CIERRE DEL DÍA
[ ] Asimiladas las diferencias entre char (Unicode) y String (texto).
[ ] Comprendido el uso de variables lógicas boolean (true/false).
[ ] Resuelto y probado el problema del buffer residual en Scanner.
[ ] Caso guía ControlAccesoQR evolucionado a la versión v0.2.
[ ] Archivo MiProyecto.java del proyecto propio actualizado a v0.2.
  
```

8. 🚀 Diario de clase: lunes, 21 de septiembre de 2026

8.1 1.º DAM — Curso académico 2026/2027

Módulos: Programación (PR) + Entornos de desarrollo (ED)

Fecha: Lunes, 21 de septiembre de 2026

Duración: 4 horas lectivas (200 min)

8.2 🕒 Sesión 1. PR - Asignación compuesta, incremento y mutabilidad de memoria

Balance de tiempo: Docente: 15 min | Estudiante: 35 min

8.2.1 1. Caso guía en AzaharTech (5 min)

- Arranque del caso guía: Pau Ferrer intenta añadir un contador de fichajes al código de la semana anterior repitiendo el nombre de la variable (`personasEnCentro = personasEnCentro + 1`).
- Alba Torres explica que en el código profesional no duplicamos variables a ambos lados del igual: se utilizan operadores compuestos y de incremento para hacer el código más compacto y reducir erratas.

8.2.2 2. Micro-exposición docente (10 min)

- Operadores de asignación compuesta en Java: cómo la expresión compacta `total += valor;` equivale a `total = total + valor;` y cómo actúa directamente sobre la celda de memoria RAM.
- Operadores unarios de incremento (`++`) y decremento (`--`).
- Distinción fundamental en memoria entre utilizar estas expresiones frente a la asignación tradicional.

8.2.3 3. Andamiaje guiado: PSeInt → Java (versión v0.4) (35 min)

Actualización conjunta a `ControlAccesoQR v0.4`. Los estudiantes abren el archivo maestro y refactorizan el código: * **Paso A.** Actualización en PSeInt (`pr/pseudocodigo/ControlAccesoQR.psc - v0.4`). * **Paso B.** Refactorización en Java (`pr/src/ControlAccesoQR.java - v0.4`). Se integran contadores para el aforo disponible y el identificador de los fichajes utilizando los operadores `+=`, `--` y `++`.

8.3 ⚙️ Sesión 2 (50 min). PR - Refactorización y aplicación al proyecto propio

Balance de tiempo: Docente: 5 min | Estudiante: 45 min

8.3.1 1. Trabajo del estudiante en el proyecto propio (40 min)

- Cada estudiante abre los archivos `.psc` y `.java` de su proyecto elegido de la bolsa de proyectos.
- Aplica la refactorización a la versión `v0.4` utilizando operadores unarios y compuestos:
- En **Aventura conversacional**: usar `--` para reducir puntos de vida o `++` para contar turnos.
- En **Motor de recomendación**: usar `+=` para sumar puntuaciones a la afinidad.
- En **Simulador de físicas 2D**: usar `+=` para acumular aceleración o distancia.
- En **Bóveda de contraseñas**: usar `--` para decrementar días restantes de caducidad.
- El docente revisa los puestos asegurándose de que nadie duplique nombres de variables en las asignaciones.

8.3.2 2. Cierre de la sesión de programación (5 min)

- Comprobación en consola de que el código compila y que el cálculo matemático funciona igual tras la refactorización (`Ctrl + Shift + F10`).

8.4 Sesión 3. ED - La mecánica interna de Git y la trazabilidad

Balance de tiempo: Docente: 25 min | Estudiante: 25 min

8.4.1 1. El caos de los «cambios varios» en AzaharTech (5 min)

- Laia Claramunt proyecta el historial de GitHub y muestra tres *commits* seguidos con mensajes inútiles: "*cambios*", "*subiendo cosas que faltaban*" y "*ahora sí que funciona*".
- Reflexión: Si el proyecto se rompe, el historial no sirve de nada. Cada confirmación debe ser una unidad atómica y documentada para evitar una caja negra opaca.

8.4.2 2. Micro-exposición docente: El modelo de datos de Git (10 min)

- Instantáneas frente a diferencias: cómo Git guarda la evolución de los archivos.
- Los tres estados locales obligatorios por los que pasa un archivo:
 - Modificado (*Modified*) — En el *Working Directory*.
 - Preparado (*Staged*) — En el *Staging Area*.
 - Confirmado (*Committed*) — En el *Local Repository*.

8.4.3 3. Inspección visual y Conventional Commits (10 min)

- Inspección de cambios: cómo leer el *Git Diff* línea a línea.
- El estándar de calidad de la industria: **Conventional Commits**. Catálogo de prefijos (`feat`, `fix`, `docs`, `refactor`) y las cuatro reglas de oro para redactar mensajes descriptivos que aporten valor.

8.5 Sesión 4. Laboratorio práctico guiado: Flujo Git en IntelliJ IDEA

Balance de tiempo: Docente: 10 min | Estudiante: 40 min

8.5.1 1. Taller de control de versiones (40 min)

Cada estudiante realiza los pasos técnicos directamente sobre el código `v0.4` que acaba de refactorizar en la clase de Programación: * **Paso 1.** Localización del panel visual de Git (`Alt + 0` o `Ctrl + K`). * **Paso 2.** Observación del estado *Modified* en los archivos `.java` y `.psc`. * **Paso 3.** Inspección visual con el visor Diff de dos columnas en el IDE. * **Paso 4.** Preparación selectiva (*Staging*) de los cambios correctos. * **Paso 5.** Redacción del Commit Convencional (ej. `refactor(pr): aplicar operadores compuestos e incrementos en cálculos de aforo`) y confirmación. * **Paso 6.** (Opcional guiado). Descarte de cambios accidentales en el Directorio de Trabajo (*Rollback*). * **Paso 7.** Sincronización final con el repositorio remoto (*Push*).

8.5.2 2. Cierre y verificación (5 min)

- El docente verifica en los perfiles web de GitHub que todos los proyectos de los alumnos tienen los *commits* sincronizados con la nomenclatura de *Conventional Commits*.

```
```text
```

## 9. 🚀 Diario de clase: jueves, 17 de septiembre de 2026

### 9.1 1.º DAM — Curso académico 2026/2027

**Módulo del día:** Programación (PR)

**Fecha:** Jueves, 17 de septiembre de 2026

**Duración:** 2 horas lectivas (100 min)

### 9.2 🕒 Sesión 1. Estándares de calidad de código, autoformateo e inspección en IntelliJ

*Balance de tiempo:* **Docente: 15 min | Estudiante: 35 min**

#### 9.2.1 1. Contexto profesional en AzaharTech (5 min)

- Alba Torres y Laia Claramunt revisan en el proyector el código de `ControlAccesoQR.java` alcanzado ayer (versión v0.3).
- El motor de cálculo de estancia funciona a la perfección, pero Laia señala detalles visuales: líneas desalineadas, llaves en posiciones inconsistentes y una advertencia amarilla en el editor sobre el objeto `Scanner` abierto.
- Alba recuerda al equipo: «*En AzaharTech no basta con que el código funcione; el código debe ser legible, seguir las convenciones oficiales de Java y quedar impecablemente estructurado antes de enviarlo al repositorio.*».

#### 9.2.2 2. Micro-exposición docente: Limpieza, indentación y comentarios (10 min)

- Convenciones de nomenclatura oficiales:**
  - PascalCase* para clases (`ControlAccesoQR`, `MiProyecto`).
  - camelCase* para variables y métodos (`minutosEstanciaTotal`, `horaEntrada`).
  - Nombres autoexplicativos: evitar variables de una sola letra como `h`, `m` o `t`.
- Indentación y formateo automático en el IDE:**
  - En Java el estándar son 4 espacios por cada bloque anidado dentro de llaves `{ }`.
  - Atajo universal de IntelliJ IDEA: `ctrl + Alt + L` (en GNU/Linux). Reorganiza y alinea todo el archivo al instante.
- Gestión de recursos:**
  - La clase `Scanner` abre un flujo de lectura del sistema operativo. Al terminar de usarlo en el `main`, debe cerrarse con `teclado.close();` para evitar advertencias de fuga de recursos (*resource leak*).
- Comentarios explicativos de línea (//):**
  - Se usan para explicar el *porqué* de una fórmula o decisión técnica, no para describir lo que ya es evidente.

ESTÁNDARES DE LIMPIEZA EN INTELIJ

Ctrl + Alt + L	Autoformatear e indentar el código
Barra lateral derecha (icono)	Visto verde: «No problems found»
<code>teclado.close();</code>	Cierre formal del canal de lectura

### 9.2.3 3. Taller guiado: Refactorización y pulido de ControlAccesoQR v0.3 (25 min)

---

*Los estudiantes abren su archivo `ControlAccesoQR.java`, aplican las herramientas de formateo y añaden comentarios técnicos explicativos:*

• **Java (pr/src/ControlAccesoQR.java - v0.3 consolidada):**

```
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 0.3: Captura completa de datos, cálculo de estancia horaria y formato limpio.
 * Módulo: Programación (PR) - Sprint 1 (RA1)
 */
import java.util.Scanner;

public class ControlAccesoQR {
 public static void main(String[] args) {
 // Canal de lectura de la entrada estándar
 Scanner teclado = new Scanner(System.in);

 // 1. Variables del terminal físico (Día 1)
 int terminalId;
 double tempVestibulo;

 // 2. Variables de identidad del usuario (Día 2)
 String dniPersona;
 String nombrePersona;
 char perfilPersona;
 boolean esEntrada;

 // 3. Variables de registro horario y estancia (Día 3)
 int horaEntrada;
 int minutoEntrada;
 int horaSalida;
 int minutoSalida;
 int minutosTotalesEntrada;
 int minutosTotalesSalida;
 int minutosEstanciaTotal;
 String tokenResumen;

 // Entrada de datos del hardware y usuario
 System.out.println("=====");
 System.out.println(" AZAHARTECH - TERMINAL DE ACCESO VESTÍBULO ");
 System.out.println(" Cliente: IES El Caminàs (Curso 2026/2027) ");
 System.out.println("=====");
 System.out.print("ID del terminal: ");
 terminalId = teclado.nextInt();

 System.out.print("Temperatura del sensor (°C): ");
 tempVestibulo = teclado.nextDouble();
 teclado.nextLine(); // Limpieza obligatoria del buffer de entrada

 System.out.print("DNI de la persona: ");
 dniPersona = teclado.nextLine();

 System.out.print("Nombre completo: ");
 nombrePersona = teclado.nextLine();

 System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita): ");
 perfilPersona = teclado.next().charAt(0);

 esEntrada = true;

 System.out.print("Hora de entrada (0-23): ");
 horaEntrada = teclado.nextInt();

 System.out.print("Minuto de entrada (0-59): ");
 minutoEntrada = teclado.nextInt();

 System.out.print("Hora de salida (0-23): ");
 horaSalida = teclado.nextInt();

 System.out.print("Minuto de salida (0-59): ");
 minutoSalida = teclado.nextInt();

 // Conversión a minutos transcurridos
 minutosTotalesEntrada = (horaEntrada * 60) + minutoEntrada;
 minutosTotalesSalida = (horaSalida * 60) + minutoSalida;

 // Cálculo directo del tiempo total de permanencia en el centro
 minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

 // Composición de cadena identificativa del registro
 tokenResumen = dniPersona + "-ESTANCIA-" + minutosEstanciaTotal;

 // Salida con formato
 System.out.println("-----");
 System.out.println("Terminal configurado: #" + terminalId);
 System.out.println("Sensor termico: " + tempVestibulo + " °C");
 System.out.println("Persona: " + nombrePersona + " (DNI: " + dniPersona + ")");
 System.out.println("Perfil: " + perfilPersona);
 System.out.println("Sentido del paso: Entrada (" + esEntrada + ")");
 System.out.println("Token: " + tokenResumen);
 System.out.println("Horario: Entrada " + horaEntrada + ":" + minutoEntrada + " | Salida " + horaSalida + ":" + minutoSalida);
 System.out.println("Permanencia total en centro: " + minutosEstanciaTotal + " minutos.");
 }
}
```

```
// Cierre del recurso para evitar advertencias del compilador
teclado.close();
}
}
```

9.2.4 4. Dinámica de inspección de código (10 min)

- Cada estudiante pulsa `ctrl + Alt + L` en su IntelliJ para observar cómo se reordenan automáticamente los márgenes y operadores.
- Revisan la esquina superior derecha del editor: debe mostrar el icono del visto en verde («*No problems found*»), acreditando que no hay advertencias de estilo.

9.3  Sesión 2. Consolidación y versionado visual del proyecto propio (v0.3)

Balance de tiempo: **Docente: 10 min | Estudiante: 40 min**

9.3.1 1. Pautas técnicas de auditoría antes del guardado (10 min)

- El docente explica los 4 criterios de aceptación que debe cumplir el archivo `MiProyecto.java` para dar por superada la primera semana:
  - a. Compilación limpia sin dependencias rotas.
  - b. Nombres de variables autoexplicativos en *camelCase*.
  - c. Cálculo secuencial con paréntesis protectores funcionando en consola.
  - d. Código autoformateado y recurso `Scanner` cerrado.

9.3.2 2. Trabajo autónomo en puesto individual (35 min)

- Cada estudiante abre su archivo único en IntelliJ ( `pr/src/MiProyecto.java` ) y en PSeInt ( `pr/pseudocodigo/MiProyecto.psc` ).
- Aplica la auditoría completa sobre su proyecto propio:
  - a. Revisa que el algoritmo de PSeInt y el código Java de la versión v0.3 son equivalentes.
  - b. Añade comentarios explicativos en la cabecera y en las operaciones de cálculo.
  - c. Ejecuta dos pruebas consecutivas en consola con datos reales de su temática para confirmar que la salida no presenta errores.
- **Subida a GitHub desde la interfaz de IntelliJ:**
  - a. Pulsa el atajo `ctrl + K` (o haz clic en el icono verde de verificación **Commit** en la barra lateral izquierda).
  - b. En el panel de Commit, marca las casillas de los archivos modificados dentro de `pr/` ( `MiProyecto.java` y `MiProyecto.psc` ).
  - c. En la caja de texto para el mensaje, escribe siguiendo el estándar convencional:

```
feat(pr): consolidar version v0.3 con captura de datos, calculos aritmeticos y limpieza de codigo
```
  - d. Despliega el botón azul inferior y selecciona «**Commit and Push**».
  - e. En la ventana de confirmación que aparece, pulsa **Push** para enviar los cambios a GitHub.

FLUJO DE GUARDADO Y SINCRONIZACIÓN EN INTELIJ	
1. Ctrl + K	→ Abre el panel visual de Commit.
2. Marcar casillas de pr/	→ Prepara los archivos (Stage).
3. Escribir mensaje convencional	→ Describe el cambio realizado.
4. Pulsar «Commit and Push»	→ Graba en local y sube a GitHub.

### 9.3.3 3. Cierre y verificación en GitHub (5 min)

- Comprobación en el navegador web de que la versión v0.3 de `MiProyecto.java` y su pseudocódigo están visibles y con su commit registrado en el repositorio remoto.

## 9.4 Estado al cierre de la jornada

### ESTADO AL CIERRE DEL DÍA

```
[] Aplicadas las convenciones de estilo oficiales (camelCase/Pascal).
[] Código autoformateado en IntelliJ mediante Ctrl + Alt + L.
[] Resuelto el cierre formal del recurso Scanner con .close().
[] Caso guía ControlAccesoQR consolidado en su versión v0.3 limpia.
[] Versión v0.3 de MiProyecto.java y pseudocódigo subidos a GitHub.
```

## 10. 🚀 Diario de clase: viernes, 18 de septiembre de 2026

### 10.1 1.º DAM — Curso académico 2026/2027

**Módulos del día:** Entornos de desarrollo (ED) · Proyecto intermodular (PI)

**Fecha:** Viernes, 18 de septiembre de 2026

**Duración:** 2 horas lectivas (100 min)

### 10.2 🧭 Sesión 1. Metodología Scrum, estructura corporativa y primer repositorio en GitHub (ED)

*Balance de tiempo:* **Docente: 15 min | Estudiante: 35 min**

#### 10.2.1 1. Contexto profesional en AzaharTech (5 min)

- Los estudiantes tienen abierto en IntelliJ IDEA el código del caso guía `ControlAccesoQR.java` y su proyecto propio compilados el jueves, junto a la captura `entorno.png` guardada en su carpeta de trabajo.
- Laia Claramunt explica que el código en un ordenador local no tiene validez profesional si no está versionado, protegido y compartido de forma transparente con el cliente y el equipo.
- Presenta el marco de trabajo ágil: en AzaharTech no se usa el modelo en cascada tradicional; se trabaja con Scrum en sprints de 3 semanas, entregando software funcional al final de cada ciclo y utilizando GitHub como canal oficial de entregas.

#### 10.2.2 2. Micro-exposición docente: El marco ágil y el repositorio digital (10 min)

- **El marco de trabajo Scrum:**
  - *Sprint:* Ciclo de desarrollo de duración fija (3 semanas en nuestro curso).
  - *Product Backlog:* Lista priorizada con todos los requisitos del cliente.
  - *Sprint Backlog:* Subconjunto de tareas comprometidas para las 3 semanas actuales.
  - *Roles:* Product Owner (representa al cliente), Scrum Master (Laia Claramunt, guía metodológica) y Developers (el equipo de desarrollo).
- **Control de versiones y repositorio remoto:**
  - *Repositorio local:* Base de datos de Git en el disco duro del alumno (`.git/`).
  - *Repositorio remoto en GitHub:* Repositorio en la nube que actúa como buzón oficial de entregas y garantiza la autoría de cada cambio.
- **El archivo `.gitignore`:**
  - Regla de higiene técnica: en Git solo se sube código fuente (`.java`, `.psc`) y documentación (`.md`, `.png`).
  - Los archivos compilados (`.class`), carpetas de salida (`out/`, `target/`) y configuraciones privadas del IDE (`.idea/`) deben quedar estrictamente excluidos.

EL FLUJO DE TRABAJO EN SCRUM		
1. Product Backlog	2. Sprint Backlog	3. Incremento
Lista global de requisitos del cliente.	Tareas seleccionadas para las 3 semanas del sprint.	Software 100% funcional.

### 10.2.3 3. Taller guiado: Creación de carpetas y primer commit visual en IntelliJ (30 min)

*Los estudiantes configuran la estructura corporativa y publican su repositorio sin usar comandos de consola:*

#### 1. Creación de la jerarquía oficial de carpetas:

- En el explorador de archivos o en IntelliJ, crean la estructura universal dentro de su espacio de trabajo:

```
azahartech/
├── equipo-alfa/
│ └── apellidos-nombre/
│ ├── README.md
│ ├── .gitignore
│ ├── ed/
│ │ └── docs/
│ │ └── entorno.png
│ ├── pr/
│ │ ├── pseudocodigo/
│ │ └── src/
│ └── pi/
│ └── docs/
└── <-- Nombre del equipo asignado
 <-- Espacio personal (sin espacios ni acentos)
 <-- Portada oficial
 <-- Archivo de exclusiones
 <-- Mueven aquí la captura del lunes
```

#### 2. Configuración del archivo `.gitignore`:

- Crean el archivo `.gitignore` en la raíz de su espacio personal con las siguientes reglas:

```
.idea/
*.iml
out/
target/
*.class
.DS_Store
```

#### 3. Vinculación con GitHub y primer commit visual:

- En IntelliJ, vinculan su cuenta de GitHub desde *Settings -> Version Control -> GitHub*.
- Inician el repositorio local desde el menú superior: *Git -> Create Git Repository...*, seleccionando su carpeta personal.
- Abren el panel visual de confirmación con **Ctrl + K** (o icono verde *Commit* en la barra lateral izquierda).
- Marcan las casillas de los archivos preparados y escriben el mensaje convencional:
 

```
feat: inicializar estructura corporativa oficial y verificar entorno con OpenJDK 21
```
- Publican el proyecto en GitHub desde el menú: *Git -> GitHub -> Share Project on GitHub*, nombrándolo `DAM-AzaharTech-Proyecto-TuNombre` y marcándolo como público.

### 10.2.4 4. Cierre y comprobación en GitHub (5 min)

- Comprobación en el navegador de que el repositorio es accesible, que la imagen `ed/docs/entorno.png` se visualiza correctamente y que el historial muestra el commit con prefijo `feat: .`

## 10.3 Sesión 2. De la idea al reto técnico: Análisis de necesidades, ODS y Sprint Backlog 1 (PI)

*Balance de tiempo:* **Docente: 10 min | Estudiante: 40 min**

### 10.3.1 1. Contexto profesional en AzaharTech (5 min)

- Laia Claramunt reúne al equipo para la sesión de Proyecto intermodular: el espacio para conectar el código con el cliente real y planificar las entregas.
- Mientras en clase se toma como referencia el control de acceso del IES El Caminàs, cada estudiante debe aterrizar los requisitos para **su proyecto elegido de la bolsa de proyectos**.

### 10.3.2 2. Micro-exposición docente: Requisitos, actores, ODS e inteligencia artificial (10 min)

- Definición del reto:** Delimitar el problema operativo del cliente y justificar por qué la digitalización aporta valor.

- **Mapa de actores:** Identificar usuarios primarios (operarios/alumnos), administradores (gestores) y sistemas externos (pantallas, terminales).
- **Alineación con los ODS:** Conectar la solución con al menos un Objetivo de Desarrollo Sostenible (ej. ODS 9: *Industria, innovación e infraestructura* u ODS 12: *Producción y consumo responsables*).
- **Uso ético y transparente de la IA (Prompt Log):**
  - La IA se utiliza como asistente de análisis y redacción, nunca como sustituto del criterio técnico.
  - Es obligatorio documentar en una tabla las herramientas empleadas, los prompts introducidos y la revisión crítica aplicada por el estudiante.

EL CAMINO DEL RETO TÉCNICO		
1. Problema real	2. Mapa actores	3. ODS e IA ética
Necesidad clara del cliente.	Quién interactúa con el software.	Impacto sostenible y registro transparente de prompts.

### 10.3.3 3. Taller guiado: Documento de análisis y creación del Sprint Backlog 1 (20 min)

*Cada estudiante trabaja en su puesto individual en su proyecto elegido:*

#### 1. Creación de carpetas de documentación y gestión:

- Dentro de `pi/`, crean las carpetas `pi/docs/` y `pi/backlog/`.

#### 2. Redacción del documento de análisis (`pi/docs/analisis-reto.md`):

- Crean el archivo con los siguientes apartados:
- *Contexto y justificación:* Problema del cliente y solución propuesta.
- *Alineación ODS:* Objetivo seleccionado y justificación en dos líneas.
- *Mapa de actores:* Tabla con perfiles de usuario y permisos en el sistema.
- *Prompt Log:* Tabla con la herramienta de IA consultada, prompt utilizado y ajuste personal aplicado.

#### 3. Creación del cuadro de tareas inicial (`pi/backlog/sprint1-backlog.md`):

- Crean el archivo con las tareas del Sprint 1 (14 sep – 2 oct) en estado pendiente ( - [ ] ):

```
Sprint Backlog 1 – [Nombre de Tu Proyecto Propio]
Periodo: 14 de septiembre – 2 de octubre de 2026
Responsable: [Tu Nombre]

📅 Cuadro de mando de tareas

Módulo: Proyecto intermodular (PI)
- [X] T-PI-01: Redactar análisis de necesidades y mapa de actores (`pi/docs/analisis-reto.md`)
- [X] T-PI-02: Registrar el uso ético de IA (*Prompt Log*)
- [X] T-PI-03: Crear el Sprint Backlog 1 inicial
- [] T-PI-04: Elaborar diagrama de bloques funcional y viabilidad técnica
- [] T-PI-05: Consolidar dossier técnico y guion de demo v0.1

Módulo: Entornos de desarrollo (ED)
- [X] T-ED-01: Instalar y verificar OpenJDK 21 e IntelliJ IDEA (`ed/docs/entorno.png`)
- [X] T-ED-02: Crear estructura corporativa oficial y repositorio en GitHub
- [] T-ED-03: Elaborar memoria técnica de marco Scrum (`ed/docs/marco-scrum.md`)
- [] T-ED-04: Auditar limpieza de repositorio y publicar tag `v0.1.0-sprint1`

Módulo: Programación (PR)
- [X] T-PR-01: Declarar variables primitivas y lectura con Scanner (`pr/src/MiProyecto.java`)
- [] T-PR-02: Implementar operadores aritméticos, módulo y casting
- [] T-PR-03: Integrar constantes `final` y salida formateada con `printf`
```

### 10.3.4 4. Guardado y sincronización visual en IntelliJ (5 min)

- Abren el panel de Commit con **ctrl + K**, seleccionan la carpeta `pi/`, escriben el mensaje: `docs(pi): definir analisis del reto del proyecto propio y sprint backlog 1` y pulsan **«Commit and Push»**.

## 10.4 Estado al cierre de la jornada

---

### ESTADO AL CIERRE DEL DÍA

```
[] Comprendido el funcionamiento de Scrum y el ciclo de sprint.
[] Creada la jerarquía corporativa azahartech/equipo-XX/alumno/.
[] Configurado el archivo .gitignore con exclusiones de binarios.
[] Repositorio público publicado en GitHub con la captura de entorno.
[] Redactado el análisis del reto y el mapa de actores en pi/docs/.
[] Registrado el uso ético de IA en la tabla de Prompt Log.
[] Inicializado el Sprint Backlog 1 con las tareas de los 3 módulos.
```

## 11. 🚀 Diario de clase: lunes, 21 de septiembre de 2026

### 11.1 1.º DAM — Curso académico 2026/2027

**Módulos:** Programación (PR) + Entornos de desarrollo (ED)

**Fecha:** Lunes, 21 de septiembre de 2026

**Duración:** 4 horas lectivas (200 min)

### 11.2 🕒 Sesión 1. PR - Asignación compuesta, incremento y mutabilidad de memoria

**Balance de tiempo:** Docente: 15 min | Estudiante: 35 min

#### 11.2.1 1. Caso guía en AzaharTech (5 min)

- Arranque del caso guía: Pau Ferrer intenta añadir un contador de fichajes al código de la semana anterior repitiendo el nombre de la variable (`personasEnCentro = personasEnCentro + 1`).
- Alba Torres explica que en el código profesional no duplicamos variables a ambos lados del igual: se utilizan operadores compuestos y de incremento para hacer el código más compacto y reducir erratas.

#### 11.2.2 2. Micro-exposición docente (10 min)

- Operadores de asignación compuesta en Java: cómo la expresión compacta `total += valor;` equivale a `total = total + valor;` y cómo actúa directamente sobre la celda de memoria RAM.
- Operadores unarios de incremento (`++`) y decremento (`--`).
- Distinción fundamental en memoria entre utilizar estas expresiones frente a la asignación tradicional.

#### 11.2.3 3. Andamiaje guiado: PSeInt → Java (versión v0.4) (35 min)

Actualización conjunta a `ControlAccesoQR v0.4`. Los estudiantes abren el archivo maestro y refactorizan el código: \* **Paso A.** Actualización en PSeInt (`pr/pseudocodigo/ControlAccesoQR.psc - v0.4`). \* **Paso B.** Refactorización en Java (`pr/src/ControlAccesoQR.java - v0.4`). Se integran contadores para el aforo disponible y el identificador de los fichajes utilizando los operadores `+=`, `-=` y `++`.

### 11.3 ⚙️ Sesión 2 (50 min). PR - Refactorización y aplicación al proyecto propio

**Balance de tiempo:** Docente: 5 min | Estudiante: 45 min

#### 11.3.1 1. Trabajo del estudiante en el proyecto propio (40 min)

- Cada estudiante abre los archivos `.psc` y `.java` de su proyecto elegido de la bolsa de proyectos.
- Aplica la refactorización a la versión `v0.4` utilizando operadores unarios y compuestos:
- En **Aventura conversacional**: usar `-=` para reducir puntos de vida o `++` para contar turnos.
- En **Motor de recomendación**: usar `+=` para sumar puntuaciones a la afinidad.
- En **Simulador de físicas 2D**: usar `+=` para acumular aceleración o distancia.
- En **Bóveda de contraseñas**: usar `--` para decrementar días restantes de caducidad.
- El docente revisa los puestos asegurándose de que nadie duplique nombres de variables en las asignaciones.

### 11.3.2 2. Cierre de la sesión de programación (5 min)

- Comprobación en consola de que el código compila y que el cálculo matemático funciona igual tras la refactorización (`Ctrl + Shift + F10`).

## 11.4 Sesión 3. ED - La mecánica interna de Git y la trazabilidad

**Balance de tiempo:** Docente: 25 min | Estudiante: 25 min

### 11.4.1 1. El caos de los «cambios varios» en AzaharTech (5 min)

- Laia Claramunt proyecta el historial de GitHub y muestra tres *commits* seguidos con mensajes inútiles: "*cambios*", "*subiendo cosas que faltaban*" y "*ahora sí que funciona*".
- Reflexión: Si el proyecto se rompe, el historial no sirve de nada. Cada confirmación debe ser una unidad atómica y documentada para evitar una caja negra opaca.

### 11.4.2 2. Micro-exposición docente: El modelo de datos de Git (10 min)

- Instantáneas frente a diferencias: cómo Git guarda la evolución de los archivos.
- Los tres estados locales obligatorios por los que pasa un archivo:
  - a. Modificado (*Modified*) — En el *Working Directory*.
  - b. Preparado (*Staged*) — En el *Staging Area*.
  - c. Confirmado (*Committed*) — En el *Local Repository*.

### 11.4.3 3. Inspección visual y Conventional Commits (10 min)

- Inspección de cambios: cómo leer el *Git Diff* línea a línea.
- El estándar de calidad de la industria: **Conventional Commits**. Catálogo de prefijos (`feat`, `fix`, `docs`, `refactor`) y las cuatro reglas de oro para redactar mensajes descriptivos que aporten valor.

## 11.5 Sesión 4. Laboratorio práctico guiado: Flujo Git en IntelliJ IDEA

**Balance de tiempo:** Docente: 10 min | Estudiante: 40 min

### 11.5.1 1. Taller de control de versiones (40 min)

Cada estudiante realiza los pasos técnicos directamente sobre el código `v0.4` que acaba de refactorizar en la clase de Programación: \* **Paso 1.** Localización del panel visual de Git (`Alt + 0` o `Ctrl + K`). \* **Paso 2.** Observación del estado *Modified* en los archivos `.java` y `.psc`. \* **Paso 3.** Inspección visual con el visor Diff de dos columnas en el IDE. \* **Paso 4.** Preparación selectiva (*Staging*) de los cambios correctos. \* **Paso 5.** Redacción del Commit Convencional (ej. `refactor(pr): aplicar operadores compuestos e incrementos en cálculos de aforo`) y confirmación. \* **Paso 6.** (Opcional guiado). Descarte de cambios accidentales en el Directorio de Trabajo (*Rollback*). \* **Paso 7.** Sincronización final con el repositorio remoto (*Push*).

### 11.5.2 2. Cierre y verificación (5 min)

- El docente verifica en los perfiles web de GitHub que todos los proyectos de los alumnos tienen los *commits* sincronizados con la nomenclatura de *Conventional Commits*.

## 11.6 Estado al cierre de la jornada

## ESTADO AL CIERRE DEL DÍA

- [ ] Comprendido el uso de operadores de incremento y asignación.
- [ ] Refactorizada la versión v0.4 del proyecto propio sin errores.
- [ ] Asimilados los 3 estados de Git (Modified, Staged, Committed).
- [ ] Entendido y aplicado el estándar de Conventional Commits.
- [ ] Cambios atómicos confirmados y enviados a GitHub (Push).

## 12. 🚀 Diario de clase: martes, 22 de septiembre de 2026

### 12.1 1.º DAM — Curso académico 2026/2027

**Módulo:** Programación (PR)

**Fecha:** Martes, 22 de septiembre de 2026

**Duración:** 2 horas lectivas (100 min)

### 12.2 🕒 Sesión 1. PR - La división entera frente al operador residuo (%)

**Balance de tiempo:** Docente: 15 min | Estudiante: 35 min

#### 12.2.1 1. Caso guía en AzaharTech (5 min)

- Arranque del caso guía: Laia Claramunt revisa el cálculo de tiempos implementado en la semana anterior. «*Actualmente el terminal calcula la estancia en minutos totales, por ejemplo, 135 minutos. Sin embargo, el equipo directivo del IES El Caminàs prefiere que los informes de acceso muestren un formato más natural y legible: horas y minutos (2 horas y 15 minutos).*»
- Pau Ferrer sugiere dividir los minutos entre 60. Alba Torres interviene para aclarar un comportamiento clave de Java: al dividir dos números enteros se pierden los decimales. Esto, que parece un error, es la herramienta perfecta para obtener las horas. Para extraer los minutos sobrantes, utilizaremos el operador residuo (%).

#### 12.2.2 2. La división entera frente al operador residuo (%) (10 min)

- **División entera (/):** Explicación de cómo funciona el motor matemático de Java cuando ambos operandos son enteros (`int`). La operación `135 / 60` no devuelve `2.25`, sino `2`. El compilador trunca la parte decimal.
- **Operador módulo o residuo (%):** Explicación de su funcionamiento al devolver el resto de una división entera. Así, `135 % 60` devuelve `15`.
- Este tándem matemático (/ y %) es el estándar en la industria del software para realizar conversiones de unidades de tiempo, distancias y divisas.

#### 12.2.3 3. Evolución a ControlAccesoQR v0.5 (35 min)

Actualización conjunta a `ControlAccesoQR v0.5`. Los estudiantes abren el archivo maestro y añaden la descomposición matemática del tiempo de estancia:

- **Paso A. Evolución en PSeInt (pr/pseudocodigo/ControlAccesoQR.psc - v0.5):**
- Se declaran las nuevas variables `horasEstancia` y `minutosEstancia` como Entero.
- Se integran las operaciones utilizando la división truncada (`trunc(minutosEstanciaTotal / 60)`) y el operador `MOD` para el resto.
- **Paso B. Evolución en Java (pr/src/ControlAccesoQR.java - v0.5):**
- Integración de las operaciones directamente en el código Java:

```
int horasEstancia = minutosEstanciaTotal / 60;
int minutosEstancia = minutosEstanciaTotal % 60;
```

- Actualización del bloque final de salida por pantalla (`System.out.println`) para formatear el mensaje: "Permanencia total en centro: " + horasEstancia + " horas y " + minutosEstancia + " minutos."

### 12.3 ⚙️ Sesión 2 (50 min). PR - Aplicación al proyecto propio

**Balance de tiempo:** Docente: 5 min | Estudiante: 45 min

### 12.3.1 1. Trabajo del estudiante: evolución de su proyecto propio (40 min)

- Cada estudiante abre los archivos `.psc` y `.java` correspondientes a la versión `v0.4` de su proyecto elegido de la bolsa de proyectos y la evoluciona a `v0.5`.
- Debe aplicar la división entera (`/`) y el operador residuo (`%`) para resolver alguna conversión en el contexto de su dominio particular:
- En **Aventura conversacional**: convertir el botín total de la partida en monedas de plata enteras y monedas de cobre restantes (`monedas / 100` y `monedas % 100`).
- En **Motor de recomendación**: distribuir el total de minutos de visualización continua (*binge-watching*) de un usuario en horas y minutos sobrantes.
- En **Simulador de físicas 2D**: convertir una distancia total recorrida en metros a un formato de kilómetros enteros y metros restantes.
- En **Bóveda de contraseñas**: transformar los días restantes para la caducidad obligatoria de la clave en semanas y días sueltos (`dias / 7` y `dias % 7`).
- El docente circula por los puestos de trabajo comprobando que no se declaren como `double` aquellas variables que por lógica de negocio deban procesarse estrictamente con división entera.

### 12.3.2 2. Cierre de la sesión de programación (10 min)

- Comprobación en la consola integrada de IntelliJ (`Ctrl + Shift + F10`) de que la lógica matemática del módulo funciona sin errores de compilación ni anomalías en los resultados impresos por pantalla.

## 12.4 Estado al cierre de la jornada

ESTADO AL CIERRE DEL DÍA	
<input type="checkbox"/>	Comprendida la diferencia entre la división entera y el residuo.
<input type="checkbox"/>	Dominio práctico del operador módulo (%) para conversiones.
<input type="checkbox"/>	Evolucionada la versión v0.5 del proyecto propio sin errores.
<input type="checkbox"/>	Lógica matemática actualizada en los diagramas de PSeInt.

## 13. 🚀 Diario de clase: miércoles, 23 de septiembre de 2026

### 13.1 1.º DAM — Curso académico 2026/2027

**Módulo:** Programación (PR)

**Fecha:** Miércoles, 23 de septiembre de 2026

**Duración:** 2 horas lectivas (100 min)

### 13.2 🕒 Sesión 1. PR - Precedencia de operadores y casting en Java

**Balance de tiempo:** Docente: 15 min | Estudiante: 35 min

#### 13.2.1 1. Caso guía en AzaharTech (5 min)

- Arranque del caso guía: Pau Ferrer quiere añadir una función al terminal para calcular el tiempo medio de permanencia de las visitas a lo largo del día. Escribe el código `double tiempoMedio = minutosTotales / totalAccesos;`. Sin embargo, al probarlo con `135` minutos y `2` accesos, el resultado por pantalla muestra `67.0` en lugar de `67.5`. ¡Se han perdido los decimales!
- Laia Claramunt interviene: «El error no está en la variable `tiempoMedio`, que es un `double`. El problema es la precedencia y cómo funciona el motor interno de Java. Al dividir dos variables enteras (`int`), Java realiza una división entera y trunca el decimal antes de guardar el resultado. Hoy aprenderemos a forzar la conversión en memoria RAM mediante el casting explícito».

#### 13.2.2 2. Precedencia de operadores y casting en Java (10 min)

- **A. Las conversiones de tipo en la memoria RAM:** Cómo Java realiza conversiones implícitas (de un tipo más pequeño a uno más grande, por ejemplo al guardar un valor `int` dentro de un `double`) y cuándo el lenguaje exige conversiones explícitas o *casting*.
- **B. La solución técnica mediante casting en divisiones:**
  - El uso de `(double)` antepuesto a una variable para convertir temporalmente su valor en la RAM a un número de coma flotante de 64 bits antes de que se ejecute la operación matemática: `(double) minutosTotales / totalAccesos;`
  - **Precedencia de operadores:** Explicación de la jerarquía de evaluación. El *casting* tiene mayor prioridad que la multiplicación y la división, que a su vez se evalúan antes que la suma y la resta.

#### 13.2.3 3. Evolución a ControlAccesoQR v0.6 (35 min)

Actualización conjunta a `ControlAccesoQR v0.6`. Los estudiantes abren el archivo maestro y añaden la lógica para calcular y mostrar métricas estadísticas con precisión decimal:

- **Paso A. Evolución en PSeInt (pr/pseudocodigo/ControlAccesoQR.psc - v0.6):**
  - Se añade la lógica algorítmica para las medias matemáticas.
- **Paso B. Evolución en Java (pr/src/ControlAccesoQR.java - v0.6):**
  - Implementación directa del casting en la fórmula de promedio:

```
double promedioEstancia = (double) minutosEstanciaTotal / contadorAccesos;
```

- Actualización del bloque de `System.out.println` final para imprimir el dato estadístico comprobando que no hay pérdida de precisión (ej. `67.5` minutos).

## 13.3 Sesión 2 (50 min). PR - Aplicación al proyecto propio

**Balance de tiempo:** Docente: 10 min | Estudiante: 40 min

### 13.3.1 1. Trabajo del estudiante: evolución de su proyecto propio (40 min)

- Cada estudiante abre los archivos `.psc` y `.java` correspondientes a su proyecto propio y los evoluciona de la versión `v0.5` a la `v0.6`.
- El objetivo central es implementar una fórmula matemática que requiera una división exacta conservando decimales, aplicando obligatoriamente el *casting* explícito `(double)`:
- En **Aventura conversacional**: calcular la media de daño infligido por turno o el ratio de precisión del jugador `((double) impactosAcertados / totalAtaques)`.
- En **Motor de recomendación**: calcular la calificación media exacta de una serie basándose en la suma total de estrellas y el número de reseñas `((double) sumaEstrellas / totalResenas)`.
- En **Simulador de físicas 2D**: calcular la velocidad media exacta dividiendo la distancia acumulada entre los ciclos o *ticks* de tiempo `((double) distanciaTotal / ticksTiempo)`.
- En **Bóveda de contraseñas**: obtener el promedio de longitud de las claves registradas frente a una métrica entera `((double) sumaLongitudes / totalClaves)`.
- El docente circula resolviendo dudas individuales sobre la correcta ubicación de los paréntesis de precedencia y el operador de *casting*.

### 13.3.2 2. Cierre de la sesión de programación (10 min)

- Ejecución de pruebas en la consola integrada de IntelliJ (`Ctrl + Shift + F10`). Cada alumno debe forzar casos de prueba impares que generen decimales exactos (ej. `5 / 2 = 2.5`) para comprobar visualmente que su sistema no imprime el temido `0.`

## 13.4 Estado al cierre de la jornada

ESTADO AL CIERRE DEL DÍA	
[ ]	Comprendida la precedencia de operadores matemáticos en Java.
[ ]	Dominado el uso del <i>casting</i> explícito <code>(double)</code> en divisiones.
[ ]	Solucionada la pérdida de precisión al operar con variables <code>int</code> .
[ ]	Evolucionada la versión <code>v0.6</code> del proyecto propio sin errores.

## 14. 🚀 Diario de clase: jueves, 24 de septiembre de 2026

### 14.1 1.º DAM — Curso académico 2026/2027

**Módulo:** Programación (PR)

**Fecha:** Jueves, 24 de septiembre de 2026

**Duración:** 2 horas lectivas (100 min)

### 14.2 🕒 Sesión 1. PR - Auditoría de calidad y consolidación v0.6

**Balance de tiempo:** Docente: 15 min | Estudiante: 35 min

#### 14.2.1 1. Caso guía en AzaharTech (5 min)

- Arranque del caso guía: Es jueves por la tarde. Concluyen las segundas 8 horas de Programación de la semana. Laia Claramunt proyecta la versión `v0.6` del código base en el repositorio.
- «Fijaos en el avance: nuestro motor matemático ya está operativo. Hemos aprendido a manipular la memoria, utilizar operadores compuestos, diferenciar la división entera del residuo y forzar la conversión de tipos. Hoy no añadiremos características nuevas; dedicaremos la sesión a auditar, asentar y documentar nuestra versión v0.6 antes de sincronizarla definitivamente con GitHub».

#### 14.2.2 2. Lista de comprobación técnico de calidad del código para la versión v0.6 (10 min)

Laia repasa en pantalla los criterios obligatorios de calidad que todo código en AzaharTech debe superar: **\* Uso correcto de operadores de asignación:** Verificación de que no se duplican variables innecesariamente (ej. utilizar `+=` o `++` para contadores). **\* Precisión en conversiones de tiempo/distancia:** Comprobación de que la separación de unidades (como horas y minutos) emplea correctamente la división entera (`/`) y el operador residuo (`%`). **\* Protección matemática mediante casting:** Auditar que las métricas estadísticas utilizan `(double)` protegiendo la operación antes de que se produzca el truncamiento decimal. **\* Autoformato de código:** Asegurar que todo el archivo pasa por el formateador del IDE (`Ctrl + Alt + L`) manteniendo la indentación a 4 espacios y que el `Scanner` se cierra correctamente (`teclado.close();`).

#### 14.2.3 3. La versión v0.6 del proyecto propio del estudiante (35 min)

- Cada estudiante abre su archivo `pr/src/MiProyecto.java` y `pr/pseudocodigo/MiProyecto.psc`.
- Audita minuciosamente su propio código aplicando la lista de comprobación de calidad expuesta.
- Asegura que los cálculos exclusivos de su dominio (Aventura conversacional, Motor de recomendación, Físicas o Contraseñas) compilan y arrojan resultados exactos en consola, mostrando decimales correctos donde se requiere.
- El docente circula por el aula validando visualmente el autoformato y resolviendo avisos (warnings) del IDE en cada puesto.

### 14.3 ⚙️ Sesión 2 (50 min). PR - Cierre del ciclo y versionado

**Balance de tiempo:** Docente: 5 min | Estudiante: 45 min

#### 14.3.1 4. Cierre formal en Git y sincronización con GitHub (45 min)

- Una vez que el código cumple los estándares de calidad, el estudiante realiza el cierre formal de la semana en su repositorio.
- **Paso 1:** Pulsa el atajo `Ctrl + K` (o el icono verde de Commit) para abrir el panel de control de versiones en IntelliJ.
- **Paso 2:** Marca las casillas de los archivos modificados dentro de la carpeta `pr/`.
- **Paso 3:** Redacta un mensaje de confirmación descriptivo utilizando el estándar de *Conventional Commits*:  
`feat(pr): consolidar version v0.6 con motor matematico, casting y operadores compuestos`

- **Paso 4:** Despliega el botón azul inferior y selecciona «**Commit and Push**».
- **Paso 5:** En la ventana final, pulsa **Push** para enviar todo su trabajo de la semana a su repositorio remoto.
- El docente verifica en el panel web de GitHub que toda la clase ha sincronizado correctamente los cambios atómicos de sus proyectos y el historial está actualizado.

## 14.4 Estado al cierre de la jornada

---

ESTADO AL CIERRE DEL DÍA	
[ ]	Código v0.6 auditado con la lista de comprobación técnica.
[ ]	Verificada la indentación y cierre de recursos (Ctrl + Alt + L).
[ ]	Cálculos del proyecto propio validados y sin pérdida de precisión.
[ ]	Sincronización exitosa con GitHub mediante Commit and Push.
[ ]	Semana 2 concluida con el motor matemático integrado.

## 15. 🚀 Diario de clase: viernes, 25 de septiembre de 2026

---

### 15.1 1.º DAM — Curso académico 2026/2027

---

**Módulos:** Entornos de desarrollo (ED) + Proyecto Intermodular (PI)

**Fecha:** Viernes, 25 de septiembre de 2026

**Duración:** 2 horas lectivas (100 min)

### 15.2 🧭 Sesión 1 (50 min). ED - El estándar Markdown en la ingeniería del software

---

**Balance de tiempo:** Docente: 15 min | Estudiante: 35 min

#### 15.2.1 1. Caso guía en AzaharTech (5 min)

---

- Arranque del caso guía: Es viernes. El código del motor matemático de la semana está validado, pero el repositorio de GitHub carece de portadas o explicaciones legibles. Alba Torres muestra un archivo `.txt` con los requisitos del IES El Caminàs que resulta confuso, y luego intenta subir un documento binario de Microsoft Word que Git no es capaz de versionar ni comparar línea a línea.
- Laia Claramunt interviene: «*Los ingenieros de software no utilizamos procesadores de texto pesados y opacos para documentar los repositorios. Utilizamos Markdown, un estándar de texto plano que permite aplicar formato semántico y estructural, y que además es renderizado de forma nativa y espectacular por GitHub. Hoy redactaremos nuestra primera memoria técnica y el panel de control del proyecto utilizando esta sintaxis.*».

#### 15.2.2 2. Sintaxis esencial de Markdown para documentación de software (10 min)

---

- **Elementos básicos:** Uso del símbolo `#` para estructurar la jerarquía de encabezados, `**` para **negritas**, y el guion `-` para crear listas de control (*checklists*).
- **Tablas en Markdown:** Cómo construir cuadrículas combinando *pipes* (`|`) y guiones (`-`) para documentar perfiles de usuario, casos de prueba o diccionarios de variables.
- **Bloques de código estructurado:** El uso de la triple comilla invertida junto al nombre del lenguaje (ej. `java`, `text`) para insertar fragmentos de código fuente formateados y legibles.

#### 15.2.3 3. Laboratorio práctico guiado: Redacción de ED-2 y panel README (35 min)

---

- **Paso 1. Redacción de la memoria técnica Scrum:** Cada estudiante crea el archivo `ed/docs/marco-scrum.md` (Entregable ED-2). En él redacta y documenta los conceptos básicos de la metodología ágil y cómo los está aplicando a su proyecto.
- **Paso 2. Transformación del README.md en panel de control:** Modificación del archivo raíz del repositorio (`README.md`) para incluir la portada del proyecto, el escudo de AzaharTech y una tabla descriptiva con la estructura de las carpetas.
- **Paso 3. Confirmación atómica y sincronización con GitHub:** El estudiante abre el panel de control de versiones (`Ctrl + K`), redacta un mensaje bajo el estándar *Conventional Commits* (ej. `docs(ed): redactar memoria tecnica scrum y actualizar panel readme`) y sube los cambios mediante *Commit and Push*.

### 15.3 ⚙️ Sesión 2 (50 min). PI - Arquitectura de bloques y viabilidad

---

**Balance de tiempo:** Docente: 20 min | Estudiante: 30 min

### 15.3.1 1. Caso guía en AzaharTech (5 min)

- Arranque del caso guía: El equipo inicia la sesión de Proyecto Intermodular. Laia proyecta en la pantalla táctil el esquema técnico del caso del IES El Caminàs, compuesto exclusivamente por tres grandes bloques conectados por flechas: captura (cámara), procesamiento (algoritmo Java) y salida (pantalla del vestíbulo).
- «La semana pasada analizamos el problema y los actores. Pero un cliente no puede financiar una idea en el aire: necesita ver el plano arquitectónico de la solución y tener la certeza de que es técnicamente viable. Hoy diseñaréis el Diagrama de Bloques Funcional de vuestro proyecto propio, evaluaréis su viabilidad contrastando fuentes oficiales de información y actualizaréis el Sprint Backlog para afrontar la recta final».

### 15.3.2 2. Fundamento metodológico: arquitectura funcional y contraste de fuentes (15 min)

- **A. El diagrama de bloques funcional (Modelo IPO):** Representación visual de muy alto nivel. Permite entender el flujo sin leer código:
- **Entradas:** Qué datos brutos se capturan por consola.
- **Procesamiento:** Qué cálculos o lógica de negocio en Java los transforman.
- **Salidas:** Qué información útil entrega el sistema por pantalla.
- **B. Estudio de viabilidad técnica:** Certificar que la solución es realizable. Evaluación del entorno de software exigido (Java OpenJDK 21 LTS e IntelliJ), requisitos mínimos de hardware del cliente (ej. mínimo 2 GB de RAM) y gestión de las restricciones operativas del Sprint 1 (procesamiento en memoria sin bases de datos relacionales).
- **C. Búsqueda y contraste de fuentes técnicas oficiales:** La regla de descartar foros anónimos o tutoriales obsoletos y acudir a fuentes primarias fiables (documentación oficial de Oracle Java SE 21).

### 15.3.3 3. Laboratorio práctico: Modelado de bloques y viabilidad técnica (30 min)

- **Paso 1. Redacción del documento de viabilidad y arquitectura:** El estudiante navega a `pi/docs/` y crea el archivo `viabilidad-tecnica.md`. Utilizando la sintaxis Markdown aprendida en la sesión anterior, dibuja un diagrama funcional mediante bloques de texto e incorpora su estudio de viabilidad.
- **Paso 2. Actualización y seguimiento del Sprint Backlog 1:** Modificación de la lista de tareas `pi/backlog/sprint1-backlog.md` para marcar con una `[x]` los hitos de programación y entornos de desarrollo conquistados a lo largo de esta segunda semana.
- **Paso 3. Confirmación y sincronización en GitHub:** Commit final empleando un prefijo convencional (ej. `docs(pi): elaborar estudio de viabilidad tecnica y actualizar sprint backlog 1`) y envío mediante *Push*.

## 15.4 Estado al cierre de la jornada

ESTADO AL CIERRE DEL DÍA	
<input type="checkbox"/>	Dominada la sintaxis básica y creación de tablas en Markdown.
<input type="checkbox"/>	Memoria Scrum (ED-2) y archivo README redactados y subidos (ED).
<input type="checkbox"/>	Diagrama funcional (IPO) y viabilidad técnica completados (PI).
<input type="checkbox"/>	Actualizado el Sprint Backlog 1 con el progreso de la Semana 2.
<input type="checkbox"/>	Semana 2 cerrada con sincronización exitosa de toda la doc técnica.

## 16. 🚀 Diario de clase: lunes, 28 de septiembre de 2026

### 16.1 1.º DAM — Curso académico 2026/2027

**Módulos:** Programación (PR) + Entornos de desarrollo (ED)

**Fecha:** Lunes, 28 de septiembre de 2026

**Duración:** 4 horas lectivas (200 min)

### 16.2 🕒 Sesión 1. PR - Constantes inmutables y literales tipados

**Balance de tiempo:** Docente: 15 min | Estudiante: 35 min

#### 16.2.1 1. Caso guía en AzaharTech (5 min)

- Arranque del caso guía: Arranca la Semana 3, la recta final del Sprint 1. Pau Ferrer revisa el código de cálculos matemáticos de la semana anterior. *«He visto que en el código fuente hemos escrito el número 60 a mano en varios sitios para convertir los minutos a horas, y también hemos fijado un límite de aforo con un 100 metido a la fuerza en el algoritmo».*
- Alba Torres interviene de inmediato: *«A eso lo llamamos 'números mágicos'. Es una mala práctica de ingeniería porque si mañana el IES El Caminàs cambia su aforo máximo, tendríamos que buscar y cambiar ese 100 línea por línea. Hoy aprenderemos a utilizar constantes inmutables y a definir los literales tipados para blindar los valores de negocio desde el inicio de la clase».*

#### 16.2.2 2. Fundamento teórico: constantes inmutables y literales tipados (10 min)

- **Constantes inmutables en Java:** Uso del modificador `final`. Una vez se le asigna un valor en memoria, la JVM bloquea la celda e impide cualquier modificación posterior. Convención de nomenclatura corporativa en AzaharTech: escritura en mayúsculas sostenidas separadas por guiones bajos ( `SNAKE_CASE` ).

```
final int AFORO_MAXIMO = 100;
final double TEMPERATURA_UMBRAL = 37.5;
```

- **Literales tipados:** Cómo el compilador de Java infiere los valores directos escritos en el código y la necesidad de sufijos para forzar tipos, como la `f` en `float` o la `L` en `long`.

#### 16.2.3 3. Refactorización a ControlAccesoQR v0.7 (35 min)

Actualización conjunta a `ControlAccesoQR v0.7`. Los estudiantes abren el archivo maestro y proceden a extraer los *números mágicos* transformándolos en constantes en la cabecera del programa:

- **Paso A. Refactorización en PSeInt ( `pr/pseudocodigo/ControlAccesoQR.psc - v0.7` ):**
- Se declaran las constantes al inicio del diagrama/código y se sustituyen los valores estáticos en las fórmulas matemáticas.
- **Paso B. Refactorización en Java ( `pr/src/ControlAccesoQR.java - v0.7` ):**
- Incorporación del bloque de constantes con el modificador `final` al inicio del `main`.
- Refactorización de las expresiones aritméticas para que utilicen los nombres de las constantes, ganando semántica y legibilidad.

### 16.3 ⚙️ Sesión 2 (50 min). PR - Aplicación al proyecto propio

**Balance de tiempo:** Docente: 10 min | Estudiante: 40 min

### 16.3.1 1. Trabajo del estudiante: refactorización de su proyecto propio (40 min)

- Cada estudiante abre los archivos `.psc` y `.java` correspondientes a la versión `v0.6` de su proyecto elegido de la bolsa de proyectos y la refactoriza a `v0.7`.
- Debe identificar valores estáticos en su lógica y extraerlos a **constantes inmutables (final)** según su dominio:
- En **Aventura conversacional**: fijar la vida máxima o el multiplicador de daño crítico (`final int VIDA_MAXIMA = 100;`).
- En **Motor de recomendación**: establecer el máximo de estrellas permitidas en una reseña (`final int ESTRELLAS_MAX = 5;`).
- En **Simulador de físicas 2D**: definir la gravedad constante o la masa por defecto (`final double GRAVEDAD_TIERRA = 9.81;`).
- En **Bóveda de contraseñas**: declarar la longitud mínima exigida o los días legales de caducidad (`final int CADUCIDAD_DIAS = 90;`).
- El docente audita el código en los distintos monitores verificando que el nombrado de las constantes cumple con el estándar estricto de mayúsculas (SNAKE\_CASE).

### 16.3.2 2. Cierre de la sesión de programación (10 min)

- Comprobación en consola (`Ctrl + Shift + F10`) verificando que la lógica matemática produce el mismo resultado que la semana anterior, asegurando que la refactorización a constantes ha sido un éxito.

## 16.4 Sesión 3. ED - El cierre del Sprint y la calidad del repositorio

**Balance de tiempo:** Docente: 25 min | Estudiante: 25 min

### 16.4.1 1. Caso guía en AzaharTech. La recta final del Sprint 1 (5 min)

- Laia Claramunt inicia la sesión de Entornos de Desarrollo: «*Esta semana se agota el tiempo de nuestro primer Sprint. El cliente espera el incremento de software funcional del sistema de acceso. En metodologías ágiles, el fin de ciclo no es solo entregar código; requiere auditar la higiene del repositorio y celebrar dos ceremonias fundamentales para la mejora continua del equipo*».

### 16.4.2 2. Las ceremonias de cierre del Sprint: Review frente a Retrospective (10 min)

- **A. La Revisión del Sprint (Sprint Review):** Sesión orientada al *producto*. Se reúne al equipo con el Product Owner y los *stakeholders* (cliente) para realizar una demostración del software construido y recoger *feedback* directo.
- **B. La Retrospectiva del Sprint (Sprint Retrospective):** Sesión técnica y humana orientada al *equipo*. Se analiza qué procesos han funcionado bien, qué herramientas han fallado y qué acciones de mejora se implementarán en el Sprint 2 para trabajar de forma más eficiente y sana.

### 16.4.3 3. Principios de higiene técnica. El control estricto de `.gitignore` (10 min)

- ¿Por qué jamás deben subirse archivos binarios o temporales al repositorio? Impacto en el rendimiento de clonado, colisiones (*merge conflicts*) en archivos compilados y exposición accidental de secretos o configuraciones de la máquina local.
- **El archivo `.gitignore`: el guardián de la limpieza:** Recordatorio del archivo que creamos en la Semana 1. Inspección de reglas mediante patrones y comodines (ej. `*.class`, `out/`).

## 16.5 Sesión 4. Laboratorio práctico guiado. Auditoría de higiene y purga del repositorio

**Balance de tiempo:** Docente: 10 min | Estudiante: 40 min

### 16.5.1 1. Auditoría de higiene y purga del repositorio (40 min)

Cada estudiante se asegura de que su repositorio esté impoluto antes del cierre oficial: \* **Paso 1. Auditoría visual de archivos en el árbol de proyectos de IntelliJ:** Detectar si accidentalmente el IDE ha marcado para seguimiento (en color verde o

blanco) archivos de la carpeta `out/` o del directorio privado `.idea/`. \* **Paso 2. Retirada visual de archivos no deseados de Git:** En caso de que se hayan colado binarios `.class`, proceder a eliminarlos del control de seguimiento (sin borrarlos físicamente del disco) mediante el comando `git rm --cached` integrado en la vista de control de versiones del IDE. \* **Paso 3. Consolidación de las reglas universales en .gitignore:** Revisión del archivo en la raíz `azahartech/nombre-equipo/.../.gitignore` garantizando que todo el directorio de compilación está formalmente ignorado. \* **Paso 4. Confirmación y sincronización visual con GitHub:** Redacción del commit final de higiene técnica (ej. `chore(ed): limpiar archivos compilados y asegurar gitignore`) y *Push* hacia el repositorio remoto.

## 16.5.2 2. Cierre y verificación de puestos (5 min)

- El docente se conecta a los repositorios web de los estudiantes para auditar rápidamente los *commits* y confirmar que nadie tiene archivos binarios publicados.

## 16.6 Estado al cierre de la jornada

ESTADO AL CIERRE DEL DÍA	
[ ]	Comprendido el uso de la palabra reservada 'final' (constantes).
[ ]	Refactorizada la versión v0.7 sustituyendo números mágicos.
[ ]	Asimiladas las diferencias entre Sprint Review y Retrospective.
[ ]	Repositorio auditado: cero archivos binarios en control de Git.
[ ]	Archivo .gitignore validado y consolidado para el cierre del Sprint

## 17. 🚀 Diario de clase: martes, 29 de septiembre de 2026

### 17.1 1.º DAM — Curso académico 2026/2027

**Módulo:** Programación (PR)

**Fecha:** Martes, 29 de septiembre de 2026

**Duración:** 2 horas lectivas (100 min)

### 17.2 🕒 Sesión 1. PR - Secuencias de escape en cadenas Java

**Balance de tiempo:** Docente: 15 min | Estudiante: 35 min

#### 17.2.1 1. Caso guía en AzaharTech (5 min)

- Arranque del caso guía: Pau Ferrer quiere mejorar la salida por consola para que el ticket de entrada sea mucho más visual. Intenta escribir comillas dobles dentro del texto para resaltar el perfil del usuario, pero el código no compila porque Java entiende que la cadena se ha cerrado. Además, quiere crear saltos de línea y tabulaciones sin usar múltiples `System.out.println`.
- Alba Torres interviene: «El compilador no puede distinguir entre las comillas que cierran una cadena y las comillas que quieres imprimir. Para eso existen las secuencias de escape: caracteres invisibles que dan instrucciones de formato a la consola y permiten incluir símbolos reservados. Hoy las aplicaremos para diseñar un ticket estructurado».

#### 17.2.2 2. Fundamento teórico: secuencias de escape en cadenas Java (10 min)

- **El carácter de escape \ (barra invertida):** Su función principal es anular el significado especial del carácter que le sigue.
- **Secuencias fundamentales en Java:**
  - `\"` : Imprime unas comillas dobles literales.
  - `\n` : Genera un salto de línea (*newline*).
  - `\t` : Genera una tabulación horizontal para alinear textos y crear columnas.
  - `\\` : Imprime una barra invertida literal (muy útil para rutas de directorios).

#### 17.2.3 3. Evolución a ControlAccesoQR v0.8 (35 min)

Actualización conjunta a `ControlAccesoQR v0.8`. Los estudiantes abren el archivo maestro y modifican el bloque de salida final por consola para formatear el resumen de acceso como un ticket oficial estructurado.

- **Paso A. Evolución en PSeInt (pr/pseudocodigo/ControlAccesoQR.psc - v0.8):**
- PSeInt maneja el formato de consola de forma distinta, pero se introducen textos espaciados para previsualizar y diseñar el ticket algorítmicamente.
- **Paso B. Evolución en Java (pr/src/ControlAccesoQR.java - v0.8):**
- Implementación de las secuencias `\n`, `\t` y `\"` dentro de los mensajes de salida, consolidando la información de manera más limpia:

```
System.out.println("\n--- TICKET DE ACCESO ---");
System.out.println("Usuario:\t\"" + nombrePersona + "\"");
System.out.println("Perfil:\t\t" + perfilPersona);
System.out.println("-----\n");
```

## 17.3 Sesión 2 (50 min). PR - Aplicación al proyecto propio

**Balance de tiempo:** Docente: 10 min | Estudiante: 40 min

### 17.3.1 1. Trabajo del estudiante: evolución de su proyecto propio (40 min)

- Cada estudiante abre los archivos `.psc` y `.java` correspondientes a la versión `v0.7` de su proyecto elegido de la bolsa de proyectos y la evoluciona a `v0.8`.
- El objetivo es rediseñar completamente la salida por pantalla utilizando las secuencias de escape (`\n`, `\t`, `\"`) para presentar un panel de datos ordenado y legible:
- En **Aventura conversacional**: formatear la hoja de personaje alineando los atributos (vida, daño) con tabulaciones e imprimiendo el nombre del avatar entre comillas.
- En **Motor de recomendación**: imprimir un ticket de reseña estructurado donde el título de la película o serie aparezca obligatoriamente entrecomillado.
- En **Simulador de físicas 2D**: diseñar un panel de telemetría alineado en dos columnas mediante tabulaciones.
- En **Bóveda de contraseñas**: simular una tabla visual en la terminal alineando los datos del servicio y resaltando la complejidad de la clave entre comillas.
- El docente circula por el aula asegurando que no se abuse de múltiples instrucciones `System.out.println()` donde una sola cadena extensa concatenada con `\n` sea más eficiente.

### 17.3.2 2. Cierre de la sesión de programación (10 min)

- Comprobación visual en la consola integrada de IntelliJ (`Ctrl + Shift + F10`). Se evalúa que las tabulaciones (`\t`) logren el efecto deseado alineando los datos como si fuesen columnas, y que las comillas literales (`\"`) se impriman sin romper la ejecución del archivo.

## 17.4 Estado al cierre de la jornada

```
ESTADO AL CIERRE DEL DÍA
[] Comprendido el uso del carácter de escape (\) y su función.
[] Dominio de las secuencias \n, \t y \" para formatear la consola.
[] Panel de salida rediseñado exitosamente en un bloque estructurado.
[] Evolucionada la versión v0.8 del proyecto propio sin errores.
```

## 18. 🚀 Diario de clase: miércoles, 30 de septiembre de 2026

### 18.1 1.º DAM — Curso académico 2026/2027

**Módulo:** Programación (PR)

**Fecha:** Miércoles, 30 de septiembre de 2026

**Duración:** 2 horas lectivas (100 min)

### 18.2 🕒 Sesión 1. PR - Especificadores de formato en System.out.printf()

**Balance de tiempo:** Docente: 15 min | Estudiante: 35 min

#### 18.2.1 1. Caso guía en AzaharTech (5 min)

- Arranque del caso guía: Pau Ferrer observa el ticket generado en la versión anterior. *«Las secuencias de escape han mejorado el diseño, pero al imprimir la temperatura del sensor térmico o el tiempo promedio de estancia, a veces salen números con demasiados decimales (ej. 21.533333333 °C), lo que rompe la alineación visual del ticket».*
- Alba Torres interviene: *«El método println() encadena cadenas y números sin control sobre su representación. Para informes profesionales y tickets, en la industria utilizamos la impresión con formato: el método System.out.printf(). Hoy dejaremos de concatenar con el símbolo + y empezaremos a inyectar variables en plantillas de texto».*

#### 18.2.2 2. Fundamento teórico: especificadores de formato en System.out.printf() (10 min)

- **El concepto de plantilla e inyección:** Cómo printf separa el texto estático de las variables, utilizando comodines (especificadores de formato) que se rellenan secuencialmente.
- **Especificadores fundamentales en Java:**
  - %d : Para números enteros ( int ).
  - %f : Para números de coma flotante ( double ). Explicación del modificador de precisión, ej. %.2f para forzar exactamente dos decimales.
  - %s : Para cadenas de texto ( String ).
  - %c : Para caracteres simples ( char ).
  - %n : El salto de línea universal, seguro en cualquier sistema operativo (preferido sobre \n dentro de printf ).

#### 18.2.3 3. Evolución a ControlAccesoQR v0.9 (35 min)

Actualización conjunta a ControlAccesoQR v0.9. Los estudiantes abren el archivo maestro y refactorizan el bloque de salida final eliminando las concatenaciones tradicionales.

- **Paso A. Evolución en PSeInt ( pr/pseudocodigo/ControlAccesoQR.psc - v0.9 ):**
- Aunque PSeInt no posee una función idéntica a printf, se asientan las bases algorítmicas de truncamiento o redondeo manual de las variables reales antes de imprimirlas para emular el control estricto de formato.
- **Paso B. Evolución en Java ( pr/src/ControlAccesoQR.java - v0.9 ):**
- Sustitución de System.out.println por System.out.printf en el bloque del ticket de acceso.
- Integración de los especificadores para redondear la temperatura a un decimal y alinear las variables:

```
System.out.printf("Temperatura del sensor: %.1f °C\n", tempVestibulo);
System.out.printf("Persona: %s (DNI: %s)\n", nombrePersona, dniPersona);
```

## 18.3 Sesión 2 (50 min). PR - Aplicación al proyecto propio

**Balance de tiempo:** Docente: 10 min | Estudiante: 40 min

### 18.3.1 1. Trabajo del estudiante: evolución de su proyecto propio (40 min)

- Cada estudiante abre los archivos `.psc` y `.java` correspondientes a la versión `v0.8` de su proyecto elegido de la bolsa de proyectos y la evoluciona a `v0.9`.
- El objetivo es eliminar cualquier concatenación engorrosa con `+` en las salidas por pantalla finales y sustituirlas íntegramente por `System.out.printf()`, aplicando máscaras de formato específicas:
- En **Aventura conversacional**: mostrar el porcentaje de salud del héroe o los multiplicadores de daño limitados a un decimal (`%.1f`).
- En **Motor de recomendación**: imprimir la nota media de la película forzada a dos decimales (`%.2f`) e inyectar el título de la obra (`%s`).
- En **Simulador de físicas 2D**: formatear la matriz de telemetría (velocidad, aceleración, masa) utilizando especificadores de número flotante para que todas las métricas queden perfectamente tabuladas.
- En **Bóveda de contraseñas**: crear un reporte de seguridad inyectando el perfil del servicio y el nivel de entropía formateado como un porcentaje numérico exacto.
- El docente circula por el aula resolviendo errores comunes, como usar accidentalmente `%d` para intentar imprimir un `double` (lo que provoca una excepción de ejecución `IllegalFormatConversionException`) o el olvido del salto de línea `%n` al final de la instrucción.

### 18.3.2 2. Cierre de la sesión de programación (10 min)

- Comprobación en la consola integrada de IntelliJ (`Ctrl + Shift + F10`). Validación de que los números con decimales "infinitos" se han redondeado visualmente y el ticket aparece alineado de manera impecable y profesional.

## 18.4 Estado al cierre de la jornada

ESTADO AL CIERRE DEL DÍA	
[ ]	Comprendida la diferencia técnica entre <code>println()</code> y <code>printf()</code> .
[ ]	Dominio de los especificadores <code>%d</code> , <code>%f</code> , <code>%s</code> , <code>%c</code> y <code>%n</code> .
[ ]	Aplicado el modificador <code>%.2f</code> para redondear salidas con decimales.
[ ]	Evolucionada la versión <code>v0.9</code> del proyecto propio sin errores.

...

## 19. 🚀 Diario de clase: jueves, 1 de octubre de 2026

---

### 19.1 1.º DAM — Curso académico 2026/2027

---

**Módulo:** Programación (PR)

**Fecha:** Jueves, 1 de octubre de 2026

**Duración:** 2 horas lectivas (100 min)

### 19.2 🕒 Sesión 1. PR - Documentación técnica y versión 1.0

---

**Balance de tiempo:** Docente: 15 min | Estudiante: 35 min

#### 19.2.1 1. Caso guía en AzaharTech (5 min)

---

- Arranque del caso guía: Es jueves por la tarde. Concluyen las sesiones de programación del Sprint 1. Laia Claramunt y Alba Torres proyectan el código final en pantalla.
- «Hemos llegado al final de nuestra primera iteración. Nuestro sistema de control de acceso por QR ya no es un prototipo, sino una aplicación de consola sólida. Pero antes de dar por cerrada esta versión (v1.0), necesitamos auditar el código. Un buen ingeniero no solo escribe código que la máquina compila, sino código estructurado y documentado que otros humanos pueden leer y mantener. Hoy finalizaremos nuestra obra».

#### 19.2.2 2. Fundamento teórico: documentación técnica y autoformato (10 min)

---

- **Documentación técnica con Javadoc:** Explicación de cómo utilizar los comentarios de bloque estructurados `/** ... */` justo encima de la cabecera de la clase y del método principal (`main`). Uso de etiquetas clave como `@author`, `@version` y la descripción del dominio.
- **Comentarios de línea:** Uso prudente de `//` para explicar "el porqué" (reglas de negocio) y no "el qué" (que debería ser evidente si las variables tienen nombres descriptivos).
- **Autoformato del IDE:** Recordatorio de la regla de oro en AzaharTech: pulsar `Ctrl + Alt + L` en IntelliJ IDEA para garantizar que la indentación a 4 espacios y los saltos de línea sean homogéneos en todo el archivo.

#### 19.2.3 3. El código definitivo del Sprint 1: ControlAccesoQR v1.0 (35 min)

---

Actualización conjunta a `ControlAccesoQR v1.0`. Los estudiantes abren el archivo maestro por última vez en este sprint:

- **Paso A. Revisión en PSeInt (pr/pseudocodigo/ControlAccesoQR.psc - v1.0):**
- Se insertan comentarios algorítmicos (`//`) al inicio del archivo resumiendo el flujo secuencial y asegurando la limpieza visual de las definiciones.
- **Paso B. Versión final en Java (pr/src/ControlAccesoQR.java - v1.0):**
- Inserción de la cabecera oficial Javadoc documentando el proyecto del IES El Caminàs.
- Ejecución del atajo de formato automático.
- Revisión general del flujo completo: desde las constantes, captura con limpieza de buffer, aritmética, hasta la inyección de salida con `printf` y secuencias de escape.

### 19.3 ⚙️ Sesión 2 (50 min). PR - Auditoría final del proyecto propio

---

**Balance de tiempo:** Docente: 10 min | Estudiante: 40 min

### 19.3.1 1. Trabajo del estudiante: la versión v1.0 de su proyecto propio (40 min)

- Cada estudiante abre los archivos `.psc` y `.java` correspondientes a su proyecto y prepara la *Release Candidate* `v1.0`.
- Aplica obligatoriamente la **lista de comprobación de calidad del código**:
- **Cabecera Javadoc:** Incluir la descripción (Aventura, Motor de recomendación, Físicas o Contraseñas), el autor (nombre del alumno) y fijar `@version 1.0`.
- **Autoformateo:** Pulsar `Ctrl + Alt + L` para eliminar tabulaciones irregulares o espacios en blanco innecesarios.
- **Constantes e inmutabilidad:** Verificar que todos los "números mágicos" han sido transformados a `final` en formato `SNAKE_CASE`.
- **Cierre de recursos:** Comprobar que `teclado.close()` está presente al final del programa sin lanzar advertencias de fuga de memoria (*resource leaks*).
- El docente circula auditando personalmente el monitor de cada estudiante para conceder el "visto bueno" técnico y aprobar su versión 1.0.

### 19.3.2 2. Cierre de la sesión y del Sprint de Programación (10 min)

- Comprobación definitiva ejecutando el proyecto propio en consola (`Ctrl + Shift + F10`) e introduciendo datos críticos reales.
- Mensaje final de cierre: el código fundacional de los cuatro proyectos ya es robusto y se encuentra listo para el inminente Sprint 2, donde la aplicación dejará de ser estrictamente secuencial y aprenderá a "tomar decisiones".

## 19.4 Estado al cierre de la jornada

ESTADO AL CIERRE DEL DÍA	
[ ]	Comprendida la importancia de documentar cabeceras con Javadoc.
[ ]	Dominado el atajo de autoformateo de IntelliJ ( <code>Ctrl + Alt + L</code> ).
[ ]	Finalizado el código fuente de ControlAccesoQR en su versión v1.0.
[ ]	Auditada y documentada la versión v1.0 del proyecto propio.
[ ]	Cierre técnico superado: el software secuencial está terminado.

...

## 20. 🚀 Diario de clase: viernes, 2 de octubre de 2026

---

### 20.1 1.º DAM — Curso académico 2026/2027

---

**Módulos:** Entornos de desarrollo (ED) + Proyecto Intermodular (PI)

**Fecha:** Viernes, 2 de octubre de 2026

**Duración:** 2 horas lectivas (100 min)

### 20.2 🕒 Sesión 1 (50 min). ED - Versionado semántico y etiquetas (TAGS)

---

**Balance de tiempo:** Docente: 15 min | Estudiante: 35 min

#### 20.2.1 1. Caso guía en AzaharTech (5 min)

---

- Arranque del caso guía: Es viernes por la mañana en AzaharTech. Laia Claramunt y Alba Torres se reúnen con el equipo. El código del sistema de control de acceso para el IES El Caminàs ( `v1.0` ) ya está purgado, autoformateado y documentado con Javadoc en la rama principal.
- «Tener el código terminado no basta. En un proyecto profesional con cientos de commits, el cliente no sabe qué momento exacto del historial corresponde a la versión final entregable. Hoy aprenderemos a "congelar" y firmar el historial mediante Etiquetas (Git Tags) y aplicaremos el estándar mundial de Versionado Semántico para empaquetar nuestra primera gran entrega».

#### 20.2.2 2. Fundamentos del versionado semántico y las etiquetas en Git (10 min)

---

- **¿Qué es un Git Tag y en qué se diferencia de una rama?** Mientras que una rama es móvil y avanza con cada nuevo commit, un Tag es una fotografía inmutable de un commit concreto. Es el equivalente a precintar una caja de software.
- **El estándar de versionado semántico (SemVer 2.0.0):** Cómo interpretar y asignar versiones siguiendo el formato Mayor.Menor.Parche (ej. `v1.0.0` o `v0.1.0`). Explicación de los sufijos de prelanzamiento (ej. `-sprint1`).

#### 20.2.3 3. Laboratorio práctico guiado: Firma de la Release y entrega oficial (35 min)

---

- **Paso 1. Cierre del panel README.md:** Cada estudiante edita la portada de su proyecto en la raíz del repositorio indicando que el Sprint 1 ha finalizado con éxito.
- **Paso 2. Creación de la etiqueta de versión formal:** Utilizando el panel Git de IntelliJ IDEA ( Log ), el estudiante localiza el último commit válido de la semana y crea una nueva etiqueta (New Tag) con el nombre corporativo obligatorio: `v0.1.0-sprint1`.
- **Paso 3. Publicación del Tag en el servidor remoto (Push Tags):** A diferencia de un push normal, los tags no viajan por defecto a GitHub. El alumno ejecuta un Push especial marcando la casilla Push Tags en el IDE.
- **Paso 4. Verificación final en GitHub:** Comprobación en la interfaz web de GitHub de que la sección Releases/Tags muestra el código empaquetado, dejando la evidencia oficial (RA4.h) lista para la calificación.

### 20.3 ⚙️ Sesión 2 (50 min). PI - Aspectos formales y técnica de comunicación oral

---

**Balance de tiempo:** Docente: 15 min | Estudiante: 35 min

#### 20.3.1 1. Caso guía en AzaharTech (5 min)

---

- Arranque del caso guía: Laia Claramunt proyecta la etiqueta de GitHub recién creada. «Equipo, el software está firmado. Pero en la vida profesional, el código no se defiende solo. El cliente del IES El Caminàs no va a leerse doscientas líneas de Java; va a valorar nuestra capacidad para documentar el producto y comunicar su valor en una demostración en vivo de tres minutos. Hoy redactaremos el Dossier Técnico y prepararemos la Sprint Review».

## 20.3.2 2. Calidad documental y comunicación técnica (10 min)

- **Aspectos formales del dossier técnico:** La documentación final debe tener estructura lógica, normas de estilo impersonales («se implementa», «el sistema calcula»), ausencia de faltas y un registro de IA (*Prompt Log*) transparente.
- **Técnicas de comunicación oral y defensa ante el cliente:**
- *The 3-Minute Demo:* Estructura de impacto.
- **Minuto 1:** Presentar al cliente y el problema.
- **Minuto 2:** Demostración en vivo en la consola de IntelliJ ejecutando datos reales.
- **Minuto 3:** Muestra del código fuente, arquitectura (diagrama IPO) y validación del Tag en GitHub.

## 20.3.3 3. Laboratorio práctico guiado: Consolidación documental y ensayo (35 min)

- **Paso 1. Creación del dossier técnico:** El estudiante crea `pi/docs/dossier-tecnico-sprint1.md` agrupando el análisis de actores, el diagrama de bloques, la viabilidad técnica y el registro de IA.
- **Paso 2. Cierre definitivo del Sprint Backlog:** Acceso a `pi/backlog/sprint1-backlog.md` para marcar el 100% de las tareas de PR, ED y PI con el check de completado [x].
- **Paso 3. Preparación del guion de la demo técnica:** Redacción final de una escaleta de 3 minutos (Minutaje de la demostración) al final del dossier.
- **Paso 4. Commit final y sincronización en GitHub:** Mensaje convencional (ej. `docs(pi): consolidar dossier tecnico final del sprint 1 y cerrar backlog al 100%`) y Push final.

20.4  Estado al cierre de la jornada y del Sprint 1

ESTADO AL CIERRE DEL SPRINT 1	
[ ]	Comprendido el Versionado Semántico (SemVer) y los Git Tags (ED).
[ ]	Repositorio firmado en GitHub con la etiqueta v0.1.0-sprint1.
[ ]	Dossier técnico PI consolidado: actores, bloques y viabilidad.
[ ]	Sprint Backlog 1 auditado y marcado al 100 % de finalización.
[ ]	Guion estructurado (3 minutos) preparado para la Sprint Review.
[ ]	¡HITOS DEL SPRINT 1 ALCANZADOS! Fin de la programación secuencial.

## 21. 🚀 Diario de clase: lunes, 5 de octubre de 2026

### 21.1 1.º DAM — Curso académico 2026/2027

**Módulos:** Programación (PR) + Entornos de desarrollo (ED)

**Fecha:** Lunes, 5 de octubre de 2026

**Duración:** 4 horas lectivas (200 min)

### 21.2 🧭 Sesión 1. PR - Fundamentos de POO, instanciación y memoria (Stack vs. Heap)

**Balance de tiempo:** Docente: 20 min | Estudiante: 30 min

#### 21.2.1 1. Caso guía en AzaharTech (5 min)

- Arranque del Sprint 2. Alba Torres proyecta el código `v1.0` cerrado la semana pasada: «*Hemos logrado que nuestro software sea un motor secuencial exacto. Pero el director del IES El Caminàs ha detectado una vulnerabilidad: el token QR usa un número correlativo simple ( #1043 , #1044 ...). Cualquiera podría predecir el siguiente código y falsificar su asistencia*».
- «*Para blindarlo, no reinventaremos la rueda con cálculos complejos; daremos el salto a la Programación Orientada a Objetos (POO). Instanciaremos clases de la biblioteca estándar de Java como `Random` y descubriremos qué ocurre bajo el capó de la memoria RAM al utilizar el operador `new`*».

#### 21.2.2 2. Fundamento teórico: clases, objetos y modelo de memoria física (15 min)

- **Clase (molde) frente a objeto (instancia):** Una clase es el plano que define comportamiento, mientras que un objeto es la entidad viva creada en memoria mediante el operador `new`.
- **Arquitectura de la memoria RAM en Java:**
- **Stack (Pila):** Memoria rápida y estructurada donde viven las variables primitivas (`int`, `double`) y las referencias (punteros a memoria).
- **Heap (Montón):** Espacio dinámico donde residen los objetos físicos reales generados con `new`.
- **El valor `null` y el temido `NullPointerException`:** Qué sucede cuando una variable de referencia en el *Stack* se declara, pero no se inicializa apuntando a un objeto real en el *Heap*.

#### 21.2.3 3. Evolución del caso guía a ControlAccesoQR v1.1 (30 min)

Actualización conjunta a `ControlAccesoQR v1.1`. Los estudiantes abren el archivo maestro para integrar la clase `Random` e inyectar un código criptográfico de seguridad:

- **Paso A. Algoritmo en PSeInt ( `pr/pseudocodigo/ControlAccesoQR.psc - v1.1` ):**
- Se añade a la lógica la variable para almacenar el código de seguridad aleatorio, aunque PSeInt no gestione el Heap como Java.
- **Paso B. Implementación en Java ( `pr/src/ControlAccesoQR.java - v1.1` ):**
- Importación del paquete `java.util.Random`.
- Instanciación del objeto: `Random generador = new Random();`
- Sustitución del identificador predecible por un número pseudoaleatorio seguro capturado con el método `generador.nextInt()`.

### 21.3 ⚙️ Sesión 2 (50 min). PR - Dojo de entrenamiento y katas de código

**Balance de tiempo:** Docente: 5 min | Estudiante: 45 min

### 21.3.1 1. Katas de código: instanciación en el proyecto propio (45 min)

Cada estudiante trabaja de forma autónoma en su proyecto propio `v1.1`, resolviendo las katas según su nivel de destreza:

- **Kata 1 (Cinturón blanco / Nivel base):** Instanciar la clase `Random` en su proyecto.
- En *Aventura*: generar tiradas de dados para calcular daño.
- En *Recomendación*: generar IDs de usuarios anónimos aleatorios.
- En *Físicas*: introducir perturbaciones aleatorias en la velocidad del viento.
- En *Contraseñas*: generar semillas para claves aleatorias.
- **Kata 2 (Cinturón marrón / Nivel avanzado):** Implementar una fórmula matemática pura para acotar los números generados a un rango estricto `[MIN, MAX]`.
- **Kata 3 (Cinturón negro / Hacker AzaharTech):** Investigación del comportamiento de la semilla (*Seed*). Instanciar el objeto pasando una semilla fija `new Random(12345L)` y explicar por qué la "aleatoriedad" se vuelve completamente determinista.

## 21.4 🗺️ Sesión 3. ED - La transformación del software y Bytecode universal

**Balance de tiempo:** Docente: 25 min | Estudiante: 25 min

### 21.4.1 1. Caso guía en AzaharTech (5 min)

- Inicia la sesión de Entornos. Laia Claramunt expone una duda del instituto: el IES El Caminàs tiene ordenadores con Windows, servidores con Linux y equipos variados. «¿*Tendremos que programar versiones distintas para cada sistema?*»
- La respuesta es negativa. Hoy entenderemos el viaje del código: cómo un archivo de texto humano se convierte en *Bytecode* y por qué la Máquina Virtual de Java hace que el software sea independiente del *hardware*.

### 21.4.2 2. La cadena de transformación y virtualización (10 min)

- **A. Código fuente (*Source code*):** El archivo `.java` legible para humanos.
- **B. Código objeto intermedio (*Bytecode*):** Archivo `.class` generado tras compilar ( `javac` ). Son instrucciones binarias para un procesador virtual genérico.
- **C. Código ejecutable (*Machine code*):** Los ceros y unos traducidos en tiempo real por la Máquina Virtual de Java (JVM) específica de cada sistema operativo (Windows, Linux, macOS). Principio *Write Once, Run Anywhere*.

### 21.4.3 3. Laboratorio práctico guiado: Inspección del archivo `.class` (10 min)

- Utilización de un visor hexadecimal para observar las tripas de `ControlAccesoQR.class` y encontrar la firma mágica `0xCAFEFABE`.
- Uso del desensamblador oficial en la terminal ( `javap -c` ) para leer el bytecode y observar las instrucciones internas ( `iload` , `imul` , `bipush` ) en las que se ha convertido el código de la sesión de PR.

## 21.5 🛠️ Sesión 4. Laboratorio práctico: Compilación multi-entorno y katas

**Balance de tiempo:** Docente: 5 min | Estudiante: 45 min

### 21.5.1 1. Katas de herramientas: la independencia del IDE (45 min)

Los estudiantes abren la terminal nativa y abandonan el confort del IDE para trabajar directamente con el *JDK*:

- **Kata 1 (Cinturón blanco / Nivel base):** Compilación y ejecución multi-entorno de su proyecto. Navegar al directorio fuente y utilizar explícitamente `javac MiProyecto.java` y `java MiProyecto`, verificando que el software funciona perfectamente sin IntelliJ.
- **Kata 2 (Cinturón marrón / Nivel avanzado):** Compilación desacoplada. Crear una carpeta limpia `bin/` y obligar al compilador a depositar allí el `.class` utilizando el parámetro de destino ( `javac -d bin ...` ). Posteriormente, arrancar la JVM apuntando la ruta mediante el parámetro de Classpath ( `java -cp bin ...` ).
- **Kata 3 (Cinturón negro / Hacker AzaharTech):** Telemetría de la JVM. Ejecutar el proyecto con la bandera de diagnóstico detallado ( `java -verbose:class` ) para observar en tiempo real la carga en memoria RAM de cientos de clases núcleo del sistema operativo antes de arrancar el propio código del estudiante.

## 21.6 Estado al cierre de la jornada

ESTADO AL CIERRE DEL DÍA	
[ ]	Comprendida la arquitectura de memoria Stack frente a Heap (PR).
[ ]	Clase Random instanciada y versión v1.1 evolucionada con éxito.
[ ]	Asimilado el flujo: Código fuente -> Bytecode -> JVM (ED).
[ ]	Archivo .class analizado mediante desensamblador javap.
[ ]	Katas de compilación manual y Classpath superadas en terminal.

...

## 22. 🚀 Diario de clase: martes, 6 de octubre de 2026

### 22.1 1.º DAM — Curso académico 2026/2027

**Módulo:** Programación (PR)

**Fecha:** Martes, 6 de octubre de 2026

**Duración:** 2 horas lectivas (100 min)

### 22.2 🧭 Sesión 1. PR - Variables primitivas frente a variables de referencia

**Balance de tiempo:** Docente: 15 min | Estudiante: 35 min

#### 22.2.1 1. Caso guía en AzaharTech (5 min)

- Arranque del caso guía: Pau Ferrer intenta crear un segundo generador aleatorio para la asignación de códigos promocionales en el vestíbulo. Escribe en el código `Random generadorPromocion = generadorAcceso;` esperando tener dos máquinas distintas. Sin embargo, al probarlo, descubre que ambos generan exactamente la misma secuencia de números, afectándose mutuamente.
- Alba Torres interviene: *«Ese es el error más común al dar el salto a los objetos. Al utilizar el operador igual (=) con variables primitivas, copias el dato real en una nueva celda de la memoria. Pero con los objetos, solo estás copiando el 'mando a distancia' (la referencia), creando un alias que apunta a la misma máquina física en el Heap. Hoy entenderemos cómo gestionar estas referencias y qué ocurre cuando un objeto se queda huérfano y actúa el Recolector de Basura».*

#### 22.2.2 2. Fundamento teórico: variables primitivas frente a variables de referencia (10 min)

- **A. El mecanismo de copia en memoria: valor frente a referencia (alias):**
  - En tipos primitivos (`int a = b;`): se clona el valor físico en dos celdas independientes del Stack.
  - En tipos de referencia (`Random a = b;`): se copia la dirección de memoria. Ambas variables se convierten en *alias* que apuntan a la misma entidad viva en el montón (Heap).
- **B. Objetos huérfanos y el recolector de basura (Garbage Collector):** Si reasignamos la única variable que apunta a un objeto hacia otro lugar, el objeto original queda "huérfano" en el Heap. La JVM detecta que nadie puede comunicarse con él y el *Garbage Collector* actúa en segundo plano para limpiar esa memoria y evitar fugas (*memory leaks*).
- **C. El valor `null` y la seguridad de referencias:** Cómo forzar que una variable de referencia suelte su objeto apuntándola conscientemente a la nada (`null`), garantizando que no modifiquemos objetos por accidente.

#### 22.2.3 3. Evolución del caso guía a ControlAccesoQR v1.2 (35 min)

Actualización conjunta a `ControlAccesoQR v1.2`. Los estudiantes abren el archivo maestro para corregir el error del alias y crear una segunda instancia real y separada en memoria:

- **Paso A. Algoritmo en PSeInt (pr/pseudocodigo/ControlAccesoQR.psc – v1.2):**
  - Actualización del diagrama introduciendo conceptualmente la lógica de los dos generadores independientes.
- **Paso B. Implementación en Java (pr/src/ControlAccesoQR.java – v1.2):**
  - Creación correcta del segundo objeto utilizando nuevamente el operador de instanciación: `Random generadorPromocion = new Random();`
  - Sustitución de la asignación defectuosa que provocaba la trampa del alias.
  - Impresión por consola de ambos resultados para verificar visualmente que las secuencias criptográficas ahora discurren de forma totalmente independiente.

## 22.3 Sesión 2 (50 min). PR - Dojo de entrenamiento y katas de código

**Balance de tiempo:** Docente: 5 min | Estudiante: 45 min

### 22.3.1 1. Segunda sesión: dojo de entrenamiento y katas de código (45 min)

Cada estudiante trabaja de forma autónoma sobre la versión `v1.2` de su proyecto elegido, resolviendo los siguientes retos de asimilación técnica:

- **Kata 1 (Cinturón blanco / Nivel base). Dos generadores independientes en tu proyecto:**
  - Instanciar explícitamente con el operador `new` dos objetos `Random` en la misma clase.
  - En *Aventura*: un generador para el daño crítico y otro para la aparición del botín.
  - En *Motor de recomendación*: un generador de identificadores de visualización y otro para inyectar ruido estadístico en las reseñas.
  - En *Simulador de físicas*: viento horizontal por un lado y turbulencias verticales por otro.
  - En *Bóveda de contraseñas*: un generador para las letras minúsculas y otro distinto para caracteres numéricos.
- **Kata 2 (Cinturón marrón / Nivel avanzado). La trampa del alias de memoria:** Forzar intencionadamente un alias (`Random copia = original;`) en el proyecto propio. Extraer valores utilizando el "original" y luego la "copia", demostrando mediante `System.out.println` cómo ambos mandos a distancia agotan la misma secuencia del objeto compartido.
- **Kata 3 (Cinturón negro / «Hacker AzaharTech»). Telemetría de objetos huérfanos y Garbage Collector:** Ensuciar intencionadamente la memoria RAM creando objetos anónimos en bucle continuo o reasignando incesantemente la misma variable de referencia. Utilizar las herramientas de la máquina virtual (como invocar `System.gc()` y medir con `Runtime.getRuntime().freeMemory()`) para documentar en la consola el momento exacto en el que el recolector de basura interviene.

## 22.4 Estado al cierre de la jornada

ESTADO AL CIERRE DEL DÍA	
[ ]	Comprendida la copia por referencia (alias) frente a por valor.
[ ]	Solucionada la trampa del alias al instanciar objetos en Java.
[ ]	Asimilado el rol del Garbage Collector limpiando el Heap.
[ ]	Evolucionada la versión v1.2 del proyecto propio sin errores.
[ ]	Katas de instanciación múltiple y memoria residual superadas.

## 23. 🚀 Diario de clase: miércoles, 7 de octubre de 2026

### 23.1 1.º DAM — Curso académico 2026/2027

**Módulo:** Programación (PR)

**Fecha:** Miércoles, 7 de octubre de 2026

**Duración:** 2 horas lectivas (100 min)

### 23.2 🕒 Sesión 1. PR - Constructores con parámetros y tipos de retorno

**Balance de tiempo:** Docente: 15 min | Estudiante: 35 min

#### 23.2.1 1. Caso guía en AzaharTech (5 min)

- Arranque del caso guía: Pau Ferrer quiere realizar simulaciones de estrés en el control de acceso del IES El Caminàs, pero los códigos aleatorios hacen que sus pruebas fallen porque cada ejecución produce números impredecibles. Además, el director solicita que se genere aleatoriamente un "factor de temperatura de error" decimal.
- Alba Torres interviene: «Hasta ahora hemos invocado clases pidiendo la configuración por defecto y extraído siempre números enteros. Pero los objetos son herramientas flexibles. Podemos inyectarles datos en el momento de nacer mediante constructores parametrizados (para obligarlos a comportarse de forma determinista) y podemos utilizar métodos que devuelven todo tipo de datos: reales, booleanos o caracteres. Hoy aprenderemos a leer las firmas de los métodos y a exprimir la clase `Random`».

#### 23.2.2 2. Fundamento teórico: constructores, firmas y tipos de retorno (10 min)

- **A. Constructores con parámetros frente a constructores vacíos:**
- El constructor vacío ( `new Random()` ) utiliza el reloj interno del sistema operativo en nanosegundos para crear la semilla de entropía.
- El constructor parametrizado ( `new Random(12345L)` ) inyecta una semilla explícita (*Seed*) al nacer. Explicación de cómo esto provoca que la secuencia "aleatoria" pase a ser estrictamente repetitiva y predecible (ideal para *testing* y auditorías).
- **B. Tipos de retorno y métodos de la clase `Random`:**
- Cómo leer e interpretar la firma de un método en la documentación oficial de Java ( `public double nextDouble()` ).
- Implementación de retornos variados: la generación de decimales con `nextDouble()` (que devuelve un valor estrictamente entre `0.0` y `1.0`) y la toma de decisiones binarias aleatorias con `nextBoolean()`.

#### 23.2.3 3. Evolución del caso guía a ControlAccesoQR v1.3 (35 min)

Actualización conjunta a `ControlAccesoQR v1.3`. Los estudiantes abren el archivo maestro para implementar constructores parametrizados para la auditoría y añadir ruido térmico decimal:

- **Paso A. Algoritmo en PSeInt ( `pr/pseudocodigo/ControlAccesoQR.psc - v1.3` ):**
- Actualización del diagrama incluyendo la simulación de variaciones térmicas (decimales) y las banderas lógicas generadas al azar.
- **Paso B. Implementación en Java ( `pr/src/ControlAccesoQR.java - v1.3` ):**
- Configuración del objeto de simulación inyectando una semilla fija: `Random auditoria = new Random(42L);`
- Extracción de ruido térmico mediante el tipo de retorno adecuado: `double errorSensor = auditoria.nextDouble();`
- Simulación de un fallo técnico aleatorio en el terminal extrayendo un valor lógico: `boolean sensorBloqueado = auditoria.nextBoolean();`

## 23.3 Sesión 2 (50 min). PR - Dojo de entrenamiento y katas de código

**Balance de tiempo:** Docente: 5 min | Estudiante: 45 min

### 23.3.1 1. Segunda sesión: dojo de entrenamiento y katas de código (45 min)

Cada estudiante trabaja de forma autónoma sobre la versión `v1.3` de su proyecto elegido, resolviendo las siguientes katas de destreza técnica:

- **Kata 1 (Cinturón blanco / Nivel base). Métodos con retorno `double` y `boolean` en tu proyecto:**
  - Invocar y guardar el resultado de métodos que devuelven distintos tipos primitivos.
  - En *Aventura*: generar una probabilidad de esquivar (`nextBoolean`) y un multiplicador de poción entre 0.0 y 1.0 (`nextDouble`).
  - En *Motor de recomendación*: determinar si un usuario es "VIP" al azar y asignar una desviación decimal en la nota de la película.
  - En *Simulador de físicas*: simular ráfagas de viento decimales y si el cuerpo de prueba colisiona en el aire de forma aleatoria.
  - En *Bóveda de contraseñas*: decidir si se incluye o no un símbolo especial en la contraseña generada (`nextBoolean`).
- **Kata 2 (Cinturón marrón / Nivel avanzado). La fábrica de números reales escalados:** Puesto que `nextDouble()` solo devuelve valores en el rango `[0.0, 1.0)`, el estudiante debe implementar una fórmula matemática utilizando sumas y multiplicaciones para lograr generar números decimales aleatorios dentro de un rango específico distinto (por ejemplo, temperaturas térmicas reales aleatorias estrictamente entre `36.0` y `40.0` °C).
- **Kata 3 (Cinturón negro / «Hacker AzaharTech»). El determinismo de la semilla en auditoría de software:** Instanciar el objeto principal del proyecto utilizando un constructor parametrizado con la semilla `new Random(1337L)`. El alumno ejecutará el código 5 veces y capturará la salida, documentando por qué el comportamiento del sistema (números y booleanos extraídos) es matemáticamente idéntico en todas las pruebas, demostrando el poder del *testing* con semillas.

## 23.4 Estado al cierre de la jornada

ESTADO AL CIERRE DEL DÍA	
[ ]	Comprendido el uso de constructores con parámetros (semillas).
[ ]	Asimiladas las firmas de los métodos y sus tipos de retorno.
[ ]	Implementados métodos <code>nextDouble()</code> y <code>nextBoolean()</code> de <code>Random</code> .
[ ]	Evolucionada la versión <code>v1.3</code> del proyecto propio sin errores.
[ ]	Katas de números escalados y determinismo criptográfico superadas.

## 24. 🚀 Diario de clase: jueves, 8 de octubre de 2026

### 24.1 1.º DAM — Curso académico 2026/2027

**Módulo:** Programación (PR)

**Fecha:** Jueves, 8 de octubre de 2026

**Duración:** 2 horas lectivas (100 min)

### 24.2 🕒 Sesión 1. PR - Métodos de transformación y extracción en la clase String

**Balance de tiempo:** Docente: 15 min | Estudiante: 35 min

#### 24.2.1 1. Caso guía en AzaharTech (5 min)

- Arranque del caso guía: Pau Ferrer está revisando los *logs* de entrada generados por los alumnos del IES El Caminàs.  
«Tenemos un problema de calidad de datos. Algunos alumnos teclean su DNI con espacios en blanco al inicio, otros escriben su nombre mezclando mayúsculas y minúsculas de forma caótica (ej. "pAu"), y a veces necesitamos extraer únicamente la primera letra de su perfil para generar el ticket. ¿Tenemos que crear bucles complejos para arreglar cada letra?»
- Alba Torres interviene con una sonrisa: «Para nada. La clase `String` en Java incluye una verdadera navaja suiza de herramientas. Hoy dejaremos de limitarnos a inspeccionar cadenas para empezar a transformarlas y trocearlas con métodos de limpieza, reemplazo y extracción. Eso sí, recordando siempre la trampa letal de la inmutabilidad de los objetos».

#### 24.2.2 2. Fundamento teórico: métodos de transformación y extracción en la clase String (10 min)

- **A. Métodos de limpieza y normalización:**
  - `trim()`: Elimina espacios en blanco o tabulaciones accidentales al principio y al final de la cadena.
  - `toUpperCase()` y `toLowerCase()`: Transformación absoluta, ideal para normalizar entradas de usuario antes de compararlas.
  - `replace(oldChar, newChar)`: Sustitución rápida de caracteres, útil para adaptar formatos (ej. cambiar comas por puntos).
- **B. Extracción de fragmentos de texto: el método `substring()`:**
  - `substring(int beginIndex)`: Devuelve una nueva cadena cortando desde el índice indicado hasta el final.
  - `substring(int beginIndex, int endIndex)`: Corta un fragmento intermedio. Explicación de la **regla estricta del `endIndex` exclusivo**: el corte se realiza justo antes del índice final, por lo que la longitud de la cadena resultante siempre es exactamente `endIndex - beginIndex`.

#### 24.2.3 3. Evolución del caso guía a ControlAccesoQR v1.8 (35 min)

Actualización conjunta a `ControlAccesoQR v1.8`. Los estudiantes abren el archivo maestro para blindar y formatear las entradas de texto introducidas en el escáner:

- **Paso A. Algoritmo en PSeInt (pr/pseudocodigo/ControlAccesoQR.psc – v1.8):**
- Implementación lógica utilizando funciones nativas como `Mayusculas(texto)` y `Subcadena(texto, inicio, fin)` para normalizar el DNI y el perfil del visitante introducido por consola.
- **Paso B. Implementación en Java (pr/src/ControlAccesoQR.java – v1.8):**
- Refactorización: Reasignación de las variables justo después de ser leídas, aplicando los métodos de limpieza en bloque:

```
dniPersona = dniPersona.trim().toUpperCase();
```

- Uso de `substring()` para generar un identificador corto en el ticket (por ejemplo, extrayendo los tres primeros caracteres del nombre y concatenándolos).

## 24.3 Sesión 2 (50 min). PR - Dojo de entrenamiento y katas de código

**Balance de tiempo:** Docente: 5 min | Estudiante: 45 min

### 24.3.1 1. Segunda sesión: dojo de entrenamiento y katas de código (45 min)

Cada estudiante trabaja de forma autónoma sobre la versión `v1.8` de su proyecto elegido, aplicando transformaciones estrictas a las cadenas de su dominio:

- **Kata 1 (Cinturón blanco / Nivel base). Transformación y extracción en tu proyecto propio:**
  - Invocar métodos para forzar el formato del texto y reasignarlo correctamente.
  - En *Aventura*: Forzar el nombre del avatar a mayúsculas y limpiarlo de espacios indeseados.
  - En *Motor de recomendación*: Extraer las tres primeras letras del título de una película ( `substring(0, 3)` ) para generar un código interno.
  - En *Simulador de físicas*: Normalizar el comando de simulación (ej. "GRAVEDAD") a minúsculas absolutas.
  - En *Bóveda de contraseñas*: Extraer el prefijo (ej. primeros 4 caracteres) del nombre del servicio.
- **Kata 2 (Cinturón marrón / Nivel avanzado). La regla del `endIndex` y el troceado dinámico:** Utilizar el método `indexOf()` aprendido en la sesión anterior para localizar un separador (por ejemplo, un espacio o un guion) y combinarlo matemáticamente con `substring()` para extraer dinámicamente la segunda palabra de un texto ingresado, sin importar su longitud.
- **Kata 3 (Cinturón negro / «Hacker AzaharTech»).** **La trampa de la inmutabilidad y el encadenamiento:** Demostrar el error común de invocar `texto.toUpperCase()`; sin guardarlo en una variable, evidenciando que el objeto original no muta en el Heap. A continuación, refactorizarlo utilizando **encadenamiento de métodos** (ej. `texto.trim().toUpperCase().replace("A", "x");` ) y documentar cuántos objetos anónimos se crean y destruyen en el *String pool* en esa única línea de código.

## 24.4 Estado al cierre de la jornada

ESTADO AL CIERRE DEL DÍA	
[ ]	Comprendido el uso de métodos de normalización: <code>trim</code> , <code>toUpperCase</code> .
[ ]	Dominada la regla del <code>endIndex</code> exclusivo en el método <code>substring()</code> .
[ ]	Asimilada la diferencia entre alterar y reasignar (inmutabilidad).
[ ]	Evolucionada la versión <code>v1.8</code> del proyecto propio sin errores.
[ ]	Katas de troceado dinámico y encadenamiento en el pool superadas.

## 25. 🚀 Diario de clase: viernes, 9 de octubre de 2026 (festivo, pero sirve para otros grupos)

---

### 25.1 1.º DAM — Curso académico 2026/2027

---

**Módulos:** Entornos de desarrollo (ED) + Proyecto Intermodular (PI)

**Fecha:** Viernes, 9 de octubre de 2026 (festivo, pero sirve para otros grupos) **Duración:** 2 horas lectivas (100 min)

### 25.2 🧭 Sesión 1 (50 min). ED - Comparación de entornos de desarrollo

---

**Balance de tiempo:** Docente: 15 min | Estudiante: 35 min

#### 25.2.1 1. Caso guía en AzaharTech (5 min)

- Arranque del caso guía: Es viernes por la tarde en la sede de AzaharTech. Laia Claramunt proyecta una consulta técnica remitida por el equipo de soporte del IES El Caminàs: «*En las aulas de informática disponen de equipos con IntelliJ IDEA, pero en los ordenadores portátiles prefieren utilizar Visual Studio Code. Nos preguntan si abrir el proyecto con otro entorno diferente puede corromper el software*».
- «Hoy comprobaremos de forma práctica que un código bien estructurado no pertenece a un fabricante de software concreto. El núcleo de Java debe comportarse con absoluta fidelidad tanto si lo editamos en un entorno libre como en un entorno propietario».

#### 25.2.2 2. El ecosistema de herramientas de desarrollo (10 min)

- **El IDE completo frente al editor modular extensible:**
- **IntelliJ IDEA Community:** Entorno integrado (IDE) completo, libre (Apache 2.0) y diseñado específicamente para la JVM. Incluye compilador, depurador y analizador de serie.
- **Visual Studio Code (VS Code):** Editor de texto modular y ligero. Utiliza extensiones (como el *Extension Pack for Java*) para comunicarse con el JDK mediante el protocolo LSP. Su instalador oficial incluye telemetría bajo licencia privativa.
- **Independencia del código fuente:** Ambos entornos utilizan bajo el capó el mismo compilador ( `javac` ) y máquina virtual ( `java` ). El código no cambia; lo que varía es la ergonomía y el consumo de recursos de la máquina.

#### 25.2.3 3. Dojo de entrenamiento y katas de herramientas (35 min)

Cada estudiante trabaja de forma autónoma con su proyecto propio para certificar su independencia del IDE:

- **Kata 1 (Cinturón blanco / Nivel base). Ejecución en un segundo entorno:** Abrir, compilar y ejecutar el archivo maestro del proyecto propio en *Visual Studio Code*, demostrando que los resultados en consola (capturas de datos, cálculos y formateos) coinciden al céntimo con los obtenidos en IntelliJ.
- **Kata 2 (Cinturón marrón / Nivel avanzado). Auditoría de consumo en RAM:** Abrir el monitor del sistema operativo (Administrador de Tareas / `htop`) y comparar la memoria consumida por IntelliJ en reposo (ej. 800 MB - 1,5 GB) frente a VS Code (ej. 300 MB - 600 MB). Anotar en la libreta técnica en qué escenarios (equipos antiguos vs. estaciones de trabajo) sería más viable cada uno.
- **Kata 3 (Cinturón negro / «Hacker AzaharTech»).** **Diagnóstico de `UnsupportedClassVersionError`:** Utilizar el desensamblador `javap -v` para rastrear la versión interna del archivo compilado ( `major version: 65` , que corresponde a Java 21) y documentar qué excepción crítica saltará si el cliente intenta ejecutar el binario en un servidor desfasado con Java 17.

### 25.3 ⚙️ Sesión 2 (50 min). PI - De la necesidad al catálogo de requisitos del sistema

---

**Balance de tiempo:** Docente: 15 min | Estudiante: 35 min

### 25.3.1 1. Caso guía en AzaharTech (5 min)

- Arranque del caso guía: El equipo arranca el módulo de Proyecto Intermodular para abrir el Sprint 2. Laia toma la palabra: «En el Sprint 1 demostramos que la lógica base era viable. Pero un cliente no firma contratos sobre intenciones generales; necesita saber con exactitud matemática qué hará y qué no hará el software».
- «Si el cliente pide que el sistema sea 'rápido y seguro', eso no le sirve a un programador. Hoy aprenderemos a transformar necesidades en un Catálogo Formal de Requisitos del Sistema y abriremos nuestro Sprint Backlog 2».

### 25.3.2 2. La ingeniería de requisitos en el desarrollo de software (10 min)

- **Requisitos Funcionales (RF) frente a No Funcionales (RNF):**
- **RF:** Describen *qué hace* el sistema (ej. capturar identificador, calcular estancia).
- **RNF:** Describen *bajo qué restricciones técnicas o de calidad* lo hace (ej. usar OpenJDK 21, estructurarse mediante Apache Maven).
- **Reglas de redacción profesional (SMART):** Todo requisito debe tener un identificador unívoco ( RF-01 ), evitar adjetivos subjetivos ("fácil", "moderno") y ser 100 % verificable mediante una prueba objetiva.
- **Fuentes técnicas y registro de IA (Prompt Log):** Obligatoriedad de contrastar información en fuentes primarias (Oracle, Apache) y documentar de forma ética cualquier consulta realizada a Inteligencia Artificial para depurar requisitos.

### 25.3.3 3. Laboratorio práctico guiado: Catálogo de requisitos y Sprint Backlog 2 (35 min)

- **Paso 1. Creación del catálogo de requisitos:** El estudiante crea el archivo `pi/docs/requisitos-sistema.md`. Redacta al menos 4 Requisitos Funcionales y 2 Requisitos No Funcionales aplicados estrictamente a su proyecto de la bolsa de proyectos (Aventura, Recomendador, Físicas o Contraseñas).
- **Paso 2. Apertura del Sprint Backlog 2:** Creación del archivo `pi/backlog/sprint2-backlog.md` listando las nuevas metas del Sprint 2 para los tres módulos técnicos.
- **Paso 3. Sincronización en GitHub:** Empleo del panel de control de versiones para realizar un commit atómico (ej. `docs(pi): definir catalogo de requisitos del sistema y abrir sprint backlog 2`) y subir la documentación técnica al repositorio remoto.

## 25.4 Estado al cierre de la jornada

```

ESTADO AL CIERRE DEL DÍA
[] Verificada la independencia del código fuente ejecutando VS Code.
[] Analizado el consumo de RAM de los diferentes IDEs del mercado.
[] Comprendida la diferencia entre un RF y un RNF verificable.
[] Creado el catálogo de requisitos del sistema (requisitos-sistema.md)
[] Abierto y sincronizado en GitHub el Sprint Backlog 2.
[] ¡SEMANA 4 COMPLETADA CON ÉXITO!

```